OVP Guide to Using Processor Models

Model specific information for RISC-V_RV64GCV

X-2025.12 December,2025

Copyright and Proprietary Information Notice

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Document Status

Author	Synopsys Inc
Version	X-2025.12
Filename	OVP_Model_Specific_Information_riscv_RV64GCV.pdf
Created	10 December 2025
Status	OVP Standard Release

Model Release Status

This model is included in all general releases.

Contents

Chapter 1

Overview

This document provides the details of an OVP Fast Processor Model variant.

OVP Fast Processor Models are written in C and provide a C API for use in C based platforms. The models also provide a native interface for use in SystemC TLM2 platforms.

The models are written using the OVP VMI API that provides a Virtual Machine Interface that defines the behavior of the processor. The VMI API makes a clear line between model and simulator allowing very good optimization and world class high speed performance. Most models are provided as a binary shared object and also as source. This allows the download and use of the model binary or the use of the source to explore and modify the model.

The models are run through an extensive QA and regression testing process and most model families are validated using technology provided by the processor IP owners. There is a companion document (OVP Guide to Using Processor Models) which explains the general concepts of OVP Fast Processor Models and their use. It is downloadable from the OVPworld website documentation pages.

1.1 Description

RISC-V RV64GCV 64-bit processor model

1.2 Licensing

This Model is released under the Open Source Apache 2.0

1.3 Extensions

1.3.1 Extensions Enabled by Default

The model has the following architectural extensions enabled, and the corresponding bits in the misa CSR Extensions field will be set upon reset:

misa bit 0: extension A (atomic instructions)

misa bit 2: extension C (compressed instructions)

misa bit 3: extension D (double-precision floating point)

misa bit 5: extension F (single-precision floating point)

misa bit 8: RV32I/RV64I/RV128I base integer instruction set

misa bit 12: extension M (integer multiply/divide instructions)

misa bit 18: extension S (Supervisor mode)

misa bit 20: extension U (User mode)

misa bit 21: extension V (vector extension)

To specify features that can be dynamically enabled or disabled by writes to the misa register in addition to those listed above, use parameter "add_Extensions_mask". This is a string parameter containing the feature letters to add; for example, value "DV" indicates that double-precision floating point and the Vector Extension can be enabled or disabled by writes to the misa register, if supported on this variant. Parameter "sub_Extensions_mask" can be used to disable dynamic update of features in the same way.

Legacy parameter "misa_Extensions_mask" can also be used. This Uns32-valued parameter specifies all writable bits in the misa Extensions field, replacing any permitted bits defined in the base variant.

Note that any features that are indicated as present in the misa mask but absent in the misa will be ignored. See the next section.

1.3.2 Enabling Other Extensions

The following misa-visible extensions are supported by the model, but not enabled by default in this variant:

misa bit 4: RV32E base integer instruction set (embedded)

misa bit 7: extension H (hypervisor)

misa bit 13: extension N (user-level interrupts)

misa bit 15: extension P (DSP instructions)

misa bit 23: extension X (non-standard extensions present)

To add features from this list to the visible set in the misa register, use parameter "add_Extensions". This is a string containing identification letters of features to enable; for example, value "DV"

indicates that double-precision floating point and the Vector Extension should be enabled, if they are currently absent and are available on this variant.

Legacy parameter "misa_Extensions" can also be used. This Uns32-valued parameter specifies the reset value for the misa CSR Extensions field, replacing any permitted bits defined in the base variant.

The following implicit extensions are supported by the model, but not enabled by default in this variant:

implicit feature bit 1: extension B (bit manipulation extension)

implicit feature bit 10: extension K (cryptographic)

To add features from this list to the implicitly-enabled set (not visible in the misa register), use parameter "add_implicit_Extensions". This is a string parameter in the same format as the "add_Extensions" parameter described above.

1.3.3 Disabling Extensions

The following misa-visible extensions are enabled by default in the model and can be disabled:

misa bit 0: extension A (atomic instructions)

misa bit 2: extension C (compressed instructions)

misa bit 3: extension D (double-precision floating point)

misa bit 5: extension F (single-precision floating point)

misa bit 12: extension M (integer multiply/divide instructions)

misa bit 18: extension S (Supervisor mode)

misa bit 20: extension U (User mode)

misa bit 21: extension V (vector extension)

To disable features that are enabled by default, use parameter "sub_Extensions". This is a string containing identification letters of features to disable; for example, value "DF" indicates that double-precision and single-precision floating point extensions should be disabled, if they are enabled by default on this variant.

1.4 General Features

1.4.1 mtvec CSR

On this variant, the Machine trap-vector base-address register (mtvec) is writable. It can instead be configured as read-only using parameter "mtvec_is_ro".

Values written to "mtvec" are masked using the value 0xfffffffffffffffd. A different mask of writable bits may be specified using parameter "mtvec_mask" if required. In addition, on a write that enables vectored interrupt mode, parameter "tvec_align" may be used to specify additional hardware-

enforced base address alignment on writes. In this variant, "tvec_align" defaults to 0, implying no additional alignment constraint on writes.

If parameter "mtvec_sext" is True, values written to "mtvec" are sign-extended from the most-significant writable bit. In this variant, "mtvec_sext" is False, indicating that "mtvec" is not sign-extended.

The initial value of "mtvec" is 0x0. A different value may be specified using parameter "mtvec" if required.

1.4.2 stvec CSR

Values written to "stvec" are masked using the value 0xfffffffffffffffd. A different mask of writable bits may be specified using parameter "stvec_mask" if required. In addition, on a write that enables vectored interrupt mode, parameter "tvec_align" may be used to specify additional hardware-enforced base address alignment on writes. In this variant, "tvec_align" defaults to 0, implying no additional alignment constraint on writes.

If parameter "stvec_sext" is True, values written to "stvec" are sign-extended from the most-significant writable bit. In this variant, "stvec_sext" is False, indicating that "stvec" is not sign-extended.

1.4.3 Reset

On reset, the model will restart at address 0x0. A different reset address may be specified using parameter "reset_address" or applied using optional input port "reset_addr" if required.

1.4.4 NMI

An NMI input is implemented. To instead specify that NMI is not implemented, set parameter "nmi_absent" to True.

On an NMI, the model will restart at address 0x0; a different NMI address may be specified using parameter "nmi_address" or applied using optional input port "nmi_addr" if required. The cause reported on an NMI is 0x0 by default; a different cause may be specified using parameter "ecode_nmi" or applied using optional input port "nmi_cause" if required.

If parameter "rnmi_version" is not "none", resumable NMIs are supported, managed by additional CSRs "mnscratch", "mnepc", "mncause" and "mnstatus", following the indicated version of the Resumable NMI extension proposal. In this variant, "rnmi_version" is "none".

On taking an NMI exception, the mstatus CSR will not be updated. To instead specify that mstatus fields mie and mpie should be updated upon taking an NMI exception, set parameter "nmi_update_mstatus" to True.

On taking an NMI exception, the tcontrol CSR will not be updated. To instead specify that tcontrol fields mte and mpte should be updated upon taking an NMI exception, set parameter "nmi_update_tcontrol" to True.

On taking an NMI exception, the mtval CSR will not be updated. To instead specify that mtval should be written with zero upon taking an NMI exception, set parameter "nmi_zero_mtval" to True.

The NMI input is level-sensitive. To instead specify that the NMI input is latched on the rising edge of the NMI signal, set parameter "nmi_is_latched" to True.

NMI interrupts are lower priority than Debug and Trigger Module events. To instead specify that NMI interrupts are higher priority, set parameter "nmi_high_priority" to True.

1.4.5 WFI

WFI will halt the processor until an interrupt occurs. It can instead be configured as a NOP by setting parameter "wfi_is_nop" to True.

The nominal time limit for WFI instructions can be set by parameter "TW_time_limit". In this variant, the time limit is 0 cycles.

Output signal "core_wfi_mode" indicates whether the processor is currently in WFI state.

Input signal "restart_wfi" will cause the processor to restart from WFI state when high.

Parameter "wfi_resume_not_trap" is 0 on this variant, meaning that pending wakeup events when WFI is executed will not prevent a trap occurring. if "wfi_resume_not_trap" is set to 1 then pending wakeup events when WFI is executed will cause the WFI to be treated as a NOP.

1.4.6 cycle CSR

The "cycle" CSR is implemented in this variant. Set parameter "cycle_undefined" to True to instead specify that "cycle" is unimplemented and accesses should cause Illegal Instruction traps.

1.4.7 instret CSR

The "instret" CSR is implemented in this variant. Set parameter "instret_undefined" to True to instead specify that "instret" is unimplemented and accesses should cause Illegal Instruction traps.

1.4.8 hpmcounter CSR

The "hpmcounter" CSRs are implemented in this variant. Set parameter "hpmcounter_undefined" to True to instead specify that "hpmcounter" CSRs are unimplemented and accesses should cause Illegal Instruction traps.

1.4.9 time CSR

The "time" CSR is implemented in this variant. Set parameter "time_undefined" to True to instead specify that "time" is unimplemented and reads of it should cause Illegal Instruction traps. Usually,

the value of the "time" CSR should be provided by the platform - see section "Time and Timers" for more information.

1.4.10 mcycle CSR

The "mcycle" CSR is implemented in this variant. Set parameter "mcycle_undefined" to True to instead specify that "mcycle" is unimplemented and accesses should cause Illegal Instruction traps.

1.4.11 minstret CSR

The "minstret" CSR is implemented in this variant. Set parameter "minstret_undefined" to True to instead specify that "minstret" is unimplemented and accesses should cause Illegal Instruction traps.

1.4.12 mhpmcounter CSR

The "mhpmcounter" CSRs are implemented in this variant. Set parameter "mhpmcounter_undefined" to True to instead specify that "mhpmcounter" CSRs are unimplemented and accesses should cause Illegal Instruction traps.

1.4.13 Virtual Memory

This variant supports address translation modes 0 (bare), 8 (Sv39), 9 (Sv48) and 10 (Sv57). Use parameter "Sv_modes" to specify a bit mask of different implemented modes if required; for example, setting "Sv_modes" to (1<<0)+(1<<8) indicates that mode 0 (bare) and mode 8 (Sv39) are implemented. These indices correspond to writable values in the satp.MODE CSR field.

A 16-bit ASID is implemented. Use parameter "ASID_bits" to specify a different implemented ASID size if required.

TLB behavior is controlled by parameter "ASIDCacheSize". If this parameter is 0, then an unlimited number of TLB entries will be maintained concurrently. If this parameter is non-zero, then only TLB entries for up to "ASIDCacheSize" different ASIDs will be maintained concurrently initially; as new ASIDs are used, TLB entries for less-recently used ASIDs are deleted, which improves model performance in some cases. If the model detects that the TLB entry cache is too small (entry ejections are very frequent), it will increase the cache size automatically. In this variant, "ASIDCacheSize" is 8.

Boolean parameter "ignore_non_leaf_DAU" specifies how non-zero D, A and U fields in non-leaf PTE entries are handled. If "ignore_non_leaf_DAU" is False, then using such entries will trigger Page Fault traps. If "ignore_non_leaf_DAU" is True, then such entries will be handled as if D, A and U fields are all zero. In this variant, "ignore_non_leaf_DAU" is 0.

1.4.14 Unaligned Accesses

Unaligned memory accesses are not supported by this variant. Set parameter "unaligned" to "T" to enable such accesses.

Unaligned memory accesses are not supported for AMO instructions by this variant. Set parameter "Zam" to "T" to enable such accesses.

Push/pop instructions will trigger address misaligned exceptions. Set parameter "unalignedPush-Pop" to "T" to enable misaligned accesses just for push/pop instructions.

Address misaligned exceptions are higher priority than page fault or access fault exceptions on this variant. Set parameter "unaligned_low_pri" to "T" to specify that they are lower priority instead.

1.4.15 PMP

16 PMP entries are implemented by this variant. Use parameter "PMP_registers" to specify a different number of PMP entries; set the parameter to 0 to disable the PMP unit.

The PMP grain size (G) is 0, meaning that PMP regions as small as 4 bytes are implemented. Use parameter "PMP_grain" to specify a different grain size if required. (The grain size in bytes is a 64 bit value calculated using the formula 4<<PMP_grain.)

Unaligned PMP accesses are not decomposed into separate aligned accesses; use parameter "PMP_decompose" to modify this behavior if required.

Parameters to change the write masks for the PMP CSRs are not enabled; use parameter "PMP_maskparams" to modify this behavior if required.

Parameters to change the reset values for the PMP CSRs are not enabled; use parameter "PMP_initialparams" to modify this behavior if required

The total number of PMP registers present is 64, (this includes CSRs for unimplemented PMP regions), as determined by the requirements of the selected priv_version. Set parameter "PMP_csrs" to specify a different total number of PMP registers.

Accesses to unimplemented PMP registers are write-ignored and read as zero on this variant. Set parameter "PMP_undefined" to True to indicate that such accesses should cause Illegal Instruction exceptions instead.

1.4.16 Time and Timers

A RISC-V hart requires an incrementing time source to be available in any of the following cases:

1. The "time" CSR is implemented ("time_undefined" is False);

2. The "Sstc" extension is present ("Sstc" is True);

3. The internal CLINT model is enabled ("CLINT_address" is non-zero).

The time value in this model is implemented by an abstract counter object, which is shared with any other processor models or peripherals that use the same name to define the counter. The name of the counter is specified by the "mtime_counter" parameter ("mtime_counter" by default). The

bit width of the counter is specified by the "mtime_bits" parameter (64 by default). The frequency of the counter is specified by the "mtime_Hz" parameter (1e+06Hz by default).

1.5 Compressed Extension

This variant implements the compressed extension with version specified in the References section of this document. Note that parameter "compress_version" can be used to select the required architecture version. See the following sections for detailed information about differences between each supported version.

1.5.1 Compressed Extension Parameters

Parameter "Zca" is used to specify that basic C extension instructions are present. By default, "Zca" is set to 1 in this variant. Updates to this parameter require a commercial product license.

Parameter "Zcf" is used to specify that floating point load/store instructions are present. By default, "Zcf" is set to 1 in this variant. Updates to this parameter require a commercial product license.

Parameter "Zcb" is used to specify that additional simple operation instructions are present. By default, "Zcb" is set to 0 in this variant. Updates to this parameter require a commercial product license.

Parameter "Zcmp" is used to specify that push/pop and double move instructions are present. By default, "Zcmp" is set to 0 in this variant. Updates to this parameter require a commercial product license.

Parameter "Zcmt" is used to specify that table jump instructions are present. By default, "Zcmt" is set to 0 in this variant. Updates to this parameter require a commercial product license.

Parameter "jvt_mask" is used to specify writable bits in the jvt CSR. By default, "jvt_mask" is set to 0xffffffffffffffc0 in this variant.

1.5.2 Legacy Version 1.10

Legacy encodings with version specified using Zcea, Zceb and Zcee parameters.

1.5.3 Version 0.70.1

All instruction encodings changed from legacy version, with instructions divided into Zca, Zcf, Zcb, Zcmb, Zcmp, Zcmpe and Zcmt subsets.

1.5.4 Version 0.70.5

Version 0.70.5, with these changes compared to version 0.70.1:

- access to jt and jalt instructions is enabled by Smstateen.

- jvt.base is WARL and fewer bits than the maximum can be implemented

1.5.5 Version 1.0.0-RC5.7

Version 1.0.0-RC5.7, with these changes compared to version 0.70.5:

- encodings of jt and jalt instructions changed.

- Zcmb and Zcmpe subsets removed.

1.5.6 Version 1.0

Ratified version 1.0, identical to version 1.0.0-RC5.7.

1.6 Floating Point Features

The D extension is enabled in this variant independently of the F extension. Set parameter "d_requires_f"to "T" to specify that the D extension requires the F extension to be enabled.

Additional floating point instructions are not implemented. Use parameter "Zfa" to enable these if required.

By default, the processor starts with floating-point instructions disabled (mstatus.FS=0). Use parameter "mstatus_FS" to force mstatus.FS to a non-zero value for floating-point to be enabled from the start.

The specification is imprecise regarding the conditions under which mstatus.FS is set to Dirty state (3). Parameter "mstatus_fs_mode" can be used to specify the required behavior in this model, as described below.

If "mstatus_fs_mode" is set to "always_dirty" then the model implements a simplified floating point status view in which mstatus.FS and .VS hold values 0 (Off) and 3 (Dirty) only; any write of values 1 (Initial) or 2 (Clean) from privileged code behave as if value 3 was written.

If "mstatus_fs_mode" is set to "write_1" then mstatus.FS will be set to 3 (Dirty) by any explicit write to the fflags, frm or fcsr control registers, or by any executed instruction that writes an FPR, or by any executed floating point compare or conversion to integer/unsigned that signals a floating point exception. Floating point compare or conversion to integer/unsigned instructions that do not signal an exception will not set mstatus.FS.

If "mstatus_fs_mode" is set to "write_any" then mstatus.FS will be set to 3 (Dirty) by any explicit write to the fflags, frm or fcsr control registers, or by any executed instruction that writes an FPR, or by any executed floating point compare or conversion even if those instructions do not signal a floating point exception.

If "mstatus_fs_mode" is set to "execute_not_store" then mstatus.FS will be set to 3 (Dirty) when any floating point instruction except a floating point store is executed. This includes all instructions selected by "write_any" mode, and also floating point moves targeting GPRs.

If "mstatus_fs_mode" is set to "execute_any" then mstatus.FS will be set to 3 (Dirty) when any floating point instruction is executed. This includes all instructions selected by "execute_not_store" mode, and also floating point stores.

In this variant, "mstatus_fs_mode" is set to "write_1".

Half precision floating point is not implemented. Use parameter "Zfh" to enable this if required.

1.7 Privileged Architecture

This variant implements the Privileged Architecture with version specified in the References section of this document. Note that parameter "priv_version" can be used to select the required architecture version; see the following sections for detailed information about differences between each supported version.

1.7.1 Legacy Version 1.10

1.10 version of May 7 2017.

1.7.2 Version 20190608

Stable 1.11 version of June 8 2019, with these changes compared to version 1.10:

- mcountinhibit CSR defined;

- pages are never executable in Supervisor mode if page table entry U bit is 1;

- mstatus.TW is writable if any lower-level privilege mode is implemented (previously, it was just if Supervisor mode was implemented);

1.7.3 Version 20211203

1.12 draft version of December 3 2021, with these changes compared to version 20190608:

- mstatush, mseccfg, mseccfgh, menvcfg, menvcfgh, senvcfg, henvcfg, henvcfgh and mconfigptr CSRs defined;

- xret instructions clear mstatus.MPRV when leaving Machine mode if new mode is less privileged than M-mode;

- maximum number of PMP registers increased to 64;

- data endian is now configurable.

1.7.4 Version 1.12

Official 1.12 version, identical to 20211203.

1.7.5 Version 1.13

1.13 ratified version of October 17, 2024, with these changes compared to version 20190608:

- misa.MXL is now defined as read-only. The MXL_writable parameter no longer exists when priv_version >= 1.13;

- Defined the RV32-only medelegh and hedelegh CSRs;

- Defined hardware-error (19) and software-check (18) exception codes;

1.7.6 Version master

Unstable master version, currently identical to 1.13.

1.8 Unprivileged Architecture

This variant implements the Unprivileged Architecture with version specified in the References section of this document. Note that parameter "user_version" can be used to select the required architecture version; see the following sections for detailed information about differences between each supported version.

1.8.1 Legacy Version 2.2

2.2 version of May 7 2017.

1.8.2 Version 20191213

Stable 20191213-Base-Ratified version of December 13 2019, with these changes compared to version 2.2:

- floating point fmin/fmax instruction behavior modified to comply with IEEE 754-201x.

- numerous other optional behaviors can be separately enabled using Z-prefixed parameters.

1.9 Vector Extension

This variant implements the RISC-V base vector extension with version specified in the References section of this document. Note that parameter "vector_version" can be used to select the required version, including the unstable "master" version corresponding to the active specification. See section "Vector Extension Versions" for detailed information about differences between each supported version.

1.9.1 Vector Extension Parameters

Parameter ELEN is used to specify the maximum size of a single vector element in bits (32 or 64). By default, ELEN is set to 64 in this variant.

Parameter VLEN is used to specify the number of bits in a vector register (a power of two in the range 32 to 65536). By default, VLEN is set to 512 in this variant.

Parameter SLEN is used to specify the striping distance (a power of two in the range 32 to 65536). By default, SLEN is set to 512 in this variant.

Parameter EEW_index is used to specify the maximum supported EEW for index load/store instructions (a power of two in the range 8 to ELEN). By default, EEW_index is set to 64 in this variant.

Parameter SEW_min is used to specify the minimum supported SEW (a power of two in the range 8 to ELEN). By default, SEW_min is set to 8 in this variant.

Parameter Zvlsseg is used to specify whether the Zvlsseg extension is implemented. By default, Zvlsseg is set to 1 in this variant.

Parameter Zvamo is used to specify whether the Zvamo extension is implemented. By default, Zvamo is set to 1 in this variant.

Parameter Zvediv will be used to specify whether the Zvediv extension is implemented. This is not currently supported.

Parameter Zvqmac is used to specify whether the Zvqmac extension is implemented (from version 0.8-draft-20191117 only). By default, Zvqmac is set to 1 in this variant.

Parameter Zve32x is used to specify whether the Zve32x extension is implemented. By default, Zve32x is set to 0 in this variant.

Parameter Zve32f is used to specify whether the Zve32f extension is implemented. By default, Zve32f is set to 0 in this variant.

Parameter Zve64x is used to specify whether the Zve64x extension is implemented. By default, Zve64x is set to 0 in this variant.

Parameter Zve64f is used to specify whether the Zve64f extension is implemented. By default, Zve64f is set to 0 in this variant.

Parameter Zve64d is used to specify whether the Zve64d extension is implemented. By default, Zve64d is set to 0 in this variant.

Parameter Zvfbfmin is used to specify whether the Zvfbfmin extension is implemented (from version 1.0 only). By default, Zvfbfmin is set to 0 in this variant.

Parameter vstart0_non_ld_st is used to specify whether non-load/store vector instructions require vstart=0. By default, vstart0_non_ld_st is set to 0 in this variant.

Parameter vstart0_non_ld_st is used to specify whether load/store vector instructions require vstart=0. By default, vstart0_ld_st is set to 0 in this variant.

Parameter align_whole is used to specify whether whole-register load and store instructions require alignment to the encoded EEW. By default, align_whole is set to 0 in this variant.

Parameter vill_trap is used to specify whether attempts to write illegal values to vtype cause an Illegal Instruction trap. By default, vill_trap is set to 0 in this variant.

Parameter agnostic_ones is used to specify whether agnostic fields are filled with all-ones (from Vector Extension version 0.9 only). By default, agnostic_ones is set to 0 in this variant, meaning mask tails, vector tail elements and vector masked-off elements all show undisturbed behavior.

Since portable code must not depend on the agnostic values in vector instruction results, it is a good idea to verify code with agnostic_ones set to True and again with it set to False. Any failure with one setting but not the other indicates the code will not be portable between vector implementations.

Parameter agnostic_mask is used to specify whether artifact agnostic mask registers va0-va31 are implemented. These artifact registers are the same size as the corresponding vector registers v0-v31. After any vector instruction with agnostic mask or tail behavior, bits in the artifact registers will be set to 1 in every position with an agnostic result in the corresponding vector register. A single 32-bit register, vaNZMask, indicates which vector agnostic mask registers currently hold non-zero values; for example, if vaNZMask is 0x00005000 then this indicates that va12 and va14 are non-zero and all other agnostic mask registers are zero. All these registers are zeroed immediately before the next instruction is executed. The registers are intended for use for design verification.

Parameter unalignedV is used to specify whether vector load and store instructions support unaligned accesses. By default, unalignedV is set to 0 in this variant, meaning unaligned accesses are not supported.

1.9.2 Vector Extension Features

The model implements the base vector extension with a maximum ELEN of 64. Striping, masking and polymorphism are all fully supported. Zvlsseg and Zvamo extensions are fully supported. The Zvediv extension specification is subject to change and therefore not yet supported.

Single precision and double precision floating point types are supported if those types are also supported in the base architecture (i.e. the corresponding D and F features must be present and enabled). Vector floating point operations may only be executed if the base floating point unit is also enabled (i.e. mstatus.FS must be non-zero). Attempting to execute vector floating point instructions when mstatus.FS is 0 will cause an Illegal Instruction exception.

The model assumes that all vector memory operations must be aligned to the memory element size. Unaligned accesses will cause a Load/Store Address Alignment exception.

By default, the processor starts with vector extension disabled (mstatus.VS=0). Use parameter "mstatus_VS" to force mstatus.VS to a non-zero value for the vector extension to be enabled from the start.

The specification is imprecise regarding the conditions under which mstatus.VS is set to Dirty state (3). Parameter "mstatus_vs_mode" can be used to specify the required behavior in this model, as described below.

If "mstatus_vs_mode" is set to "state_update" then mstatus.FS will be set to 3 (Dirty) when a vector instruction that can update state is executed. This does NOT include instructions executed with vl=0, whichcannot update any vector state.

If "mstatus_fs_mode" is set to "execute_any" then mstatus.VS will be set to 3 (Dirty) when any vector instruction is executed, even if vl=0.

Note that the behavior of mstatus.VS is also affected when mstatus_fs_mode is set to "always_dirty" in that mstatus.VS can hold values 0 (Off) and 3 (Dirty) only; any write of values 1 (Initial) or 2 (Clean) from privileged code behave as if value 3 was written.

1.9.3 Vector Extension Versions

The Vector Extension specification has been under active development. To enable simulation of hardware that may be based on an older version of the specification, the model implements behavior for a number of previous versions of the specification. The differing features of these are listed below, in chronological order.

1.9.4 Version 0.7.1-draft-20190605

Stable 0.7.1 version of June 10 2019 (commit 9dbf12d).

1.9.5 Version 0.7.1-draft-20190605+

Version 0.7.1, with some 0.8 and custom features. Not intended for general use.

1.9.6 Version 0.8-draft-20190906

Stable 0.8 draft of September 6 2019 (commit be942b4), with these changes compared to version 0.7.1-draft-20190605:

- tail vector and scalar elements preserved, not zeroed;

- vext.s.v, vmford.vv and vmford.vf instructions removed;

- encodings for vfmv.f.s, vfmv.s.f, vmv.s.x, vpopc.m, vfirst.m, vmsbf.m, vmsif.m, vmsof.m, viota.m and vid.v instructions changed;

- overlap constraints for slideup and slidedown instructions relaxed to allow overlap of destination and mask when SEW=1;

- 64-bit vector AMO operations replaced with SEW-width vector AMO operations;

- vsetvl and vsetvli instructions when rs1 = x0 preserve the current vl instead of selecting the maximum possible vl;

- instruction vfncvt.rod.f.f.w added (to allow narrowing floating point conversions with jamming semantics);

- instructions that transfer values between vector registers and general purpose registers (vmv.s.x and vmv.x.s) sign-extend the source if required (previously, it was zero-extended) (commit b9fd7c9).

1.9.7 Version 0.8-draft-20191004

Stable 0.8 draft of October 4 2019 (commit 55bb8f6), with these changes compared to version 0.8-draft-20190906:

- vwmaccsu and vwmaccus instruction encodings exchanged;

- vwsmaccsu and vwsmaccus instruction encodings exchanged.

1.9.8 Version 0.8-draft-20191117

Stable 0.8 draft of November 17 2019 (commit 3cb0a35), with these changes compared to version 0.8-draft-20191004:

- indexed load/store instructions zero-extend offsets (previously, they were sign-extended) (commit 8d4492e);

- vslide1up/vslide1down instructions sign-extend XLEN values to SEW length (previously, they were zero-extended) (commit d06438e);

- vadc/vsbc instruction encodings require vm=0 (previously, they required vm=1) (commit 5a038da);

- vmadc/vmsbc instruction encodings allow both vm=0, implying carry input is used, and vm=1, implying carry input is zero (previously, only vm=1 was permitted, implying carry input is used) (commit 5a038da);

- vaaddu.vv, vaaddu.vx, vasubu.vv and vasubu.vx instructions added (commit c2f3157);

- vaadd.vv and vaadd.vx, instruction encodings changed (commit c2f3157);

- vaadd.vi instruction removed (commit c2f3157);

- all widening saturating scaled multiply-add instructions removed (commit 063b128);

- vqmaccu.vv, vqmaccu.vx, vqmacc.vv, vqmacc.vx, vqmacc.vx, vqmaccsu.vx and vqmaccus.vx instructions added (commit 200a557);

- CSR vlenb added (vector register length in bytes) (commit 7b02297);

- load/store whole register instructions added (commit 7b02297);

- whole register move instructions added (commit 7b02297).

1.9.9 Version 0.8-draft-20191118

Stable 0.8 draft of November 18 2019 (commit 812001b), with these changes compared to version 0.8-draft-20191117:

- vsetvl/vsetvli with rd!=zero and rs1=zero sets vl to the maximum vector length (commit b6c48c3).

1.9.10 Version 0.8

Stable 0.8 official release (commit 9a65519), with these changes compared to version 0.8-draft-20191118:

- vector context status in mstatus register is now implemented (commit a6f94e7);

- whole register load and store operations have been restricted to a single register only (commit 49cbd95);

- whole register move operations have been restricted to aligned groups of 1, 2, 4 or 8 registers only (commit 49cbd9).

1.9.11 Version 0.9

Stable 0.9 official release (commit cb7d225), with these significant changes compared to version 0.8:

- mstatus.VS and sstatus.VS fields moved to bits 10:9 (commit bdb8b55);

- new CSR vcsr added and fields VXSAT and VXRM relocated there from CSR fcsr (commit b25b643 and 951b64f);

- vfslide1up.vf, vfslide1down.vf, vfcvt.rtz.xu.f.v, vfcvt.rtz.x.f.v, vfwcvt.rtz.xu.f.v, vfwcvt.rtz.x.f.v, vfncvt.rtz.xu.f.v, vfncvt.rtz.x.f.v, vzext.vf2, vsext.vf2, vzext.vf4, vsext.vf4, vzext.vf8 and vsext.vf8 instructions added (commit 1aceea2, e256d65 and bdc85cd);

- fractional LMUL support added, controlled by an extended vtype.vlmul CSR field (commit 8a9fbce);

- vector tail agnostic and vector mask agnostic fields added to the vtype CSR (commit f414f4d and others);

- all vector load/store instructions replaced with new instructions that explicitly encode EEW of data or index(commit a526fb9 and others);

- whole register load and store operation encodings changed (commit ef531ea);

- VFUNARY0 and VFUNARY1 encodings changed;

- MLEN is always 1 (commit 9a77e12);

- for implementations with SLEN != VLEN, striping is applied horizontally rather than the previous vertical striping;

- vmsbf.m, vmsif.m and vmsof.m no longer allow overlap of destination with source or mask registers.

1.9.12 Version 1.0-draft-20210130

Stable 1.0-draft-20210130 official release (commit 8e768b0), with these changes compared to version 0.9:

- SLEN=VLEN register layout is mandatory (commit 7facdcc);

- ELEN>VLEN is now supported for LMUL>1 (commit b454810);

- whole register moves and load/stores now have element size hints (commit 20f673c);

- whole register load and store operations now permit use of aligned groups of 1, 2, 4 or 8 registers.

- overlap constraints for different source/destination EEW changed (commit 443ce5b);

- instructions vfrsqrt7.v, vfrec7.v and vrgatherei16.vv added (commit d35b23f and a679250);

- CSR vtype format changed to make vlmul bits contiguous (commit b8cd98b);

- vsetvli x0, x0, imm instruction is reserved if it would cause vl to change (commit 579fef9);

- ordered/unordered indexed vector memory instructions added (commit 511d0b8);

- instructions vle1.v, vse1.v and vsetivli added (commit 7cf5349 and 8bf26e8).

1.9.13 Version 1.0-rc1-20210608

Stable 1.0-rc1-20210608 official release (commit 795a4dd), with these changes compared to version 1.0-draft-20210130:

- instructions vle1.v/vse1.v renamed vlm.v/vsm.v (commit 4ab6506);

- instructions vfredsum.vs/vfwredsum.vs renamed vfredusum.vs/vfwredusum.vs (commit a66753c);

- whole-register load/store instructions now use the EEW encoded in the instruction to determine element size (previously, this was a hint and element size 8 was used) (commit c841730);

- misa.V explicitly depends on misa.F and misa.D, if those bits are present.

1.9.14 Version 1.0

Stable 1.0 official release (commit 8af318f), with these changes compared to version 1.0-rc1-20210608:

- instruction vpopc.m renamed vcpop.m (commit 8fab4e3);

- instruction vmandnot.mm renamed vmandn.mm (commit c027517);

- instruction vmornot.mm renamed vmorn.mm (commit c027517).

1.9.15 Version 1.0-ratified

Ratified 1.0 version from RISC-V Instruction Set Manual Volume I Unprivileged Architecture Version 20240411 (github.com/riscv/riscv-isa-manual commit 6ee57d8), with these changes compared to version 1.0:

- instruction encodings are reserved if the same vector register would be read with two or more different EEWs (commit ef8b3a4).

1.9.16 Version master

Unstable master version as of 28 May 2024 (commit github.com/riscv/riscv-isa-manual tag riscv-isa-release-1bec7d3-2024-05-28), with these changes compared to version 1.0-ratified (subject to change):

- None to date

1.10 Other Extensions

Other extensions that can be configured are described in this section.

1.10.1 Zmmul

Parameter "Zmmul" is 0 on this variant, meaning that all multiply and divide instructions are implemented. if "Zmmul" is set to 1 then multiply instructions are implemented but divide and remainder instructions are not implemented.

1.10.2 Zicsr

Parameter "Zicsr" is 1 on this variant, meaning that standard CSRs and CSR access instructions are implemented. if "Zicsr" is set to 0 then standard CSRs and CSR access instructions are not implemented and an alternative scheme must be provided as a processor extension.

1.10.3 Zifencei

Parameter "Zifencei" is 1 on this variant, meaning that the fence.i instruction is implemented (but treated as a NOP by the model). if "Zifencei" is set to 0 then the fence.i instruction is not implemented.

1.10.4 Zihintpause

Parameter "Zihintpause" is 1 on this variant, meaning that the pause instruction is implemented. When executed and the PAUSE_time_limit parameter is not 0 it will pause the hart in WFI mode for up to the number of cycles specified by the time limit. When the time limit is 0 PAUSE is a NOP. if "Zihintpause" is set to 0 then the pause instruction is not implemented - instead the opcode will be interpreted as a FENCE instruction.

1.10.5 Zicbom

Parameter "Zicbom" is 0 on this variant, meaning that code block management instructions are undefined. if "Zicbom" is set to 1 then code block management instructions cbo.clean, cbo.flush and cbo.inval are defined.

If Zicbom is present, the cache block size is given by parameter "cmomp_bytes". The instructions may cause traps if used illegally but otherwise are NOPs in this model.

1.10.6 Zicbop

Parameter "Zicbop" is 0 on this variant, meaning that prefetch instructions are undefined. if "Zicbop" is set to 1 then prefetch instructions prefetch.i, prefetch.r and prefetch.w are defined (but behave as NOPs in this model).

1.10.7 Zicboz

Parameter "Zicboz" is 0 on this variant, meaning that the cbo.zero instruction is undefined. if "Zicboz" is set to 1 then the cbo.zero instruction is defined.

If Zicboz is present, the cache block size is given by parameter "cmoz_bytes".

1.10.8 Svnapot

Parameter "Svnapot_page_mask" is 0x0 on this variant, meaning that NAPOT Translation Contiguity is not implemented. if "Svnapot_page_mask" is non-zero then NAPOT Translation Contiguity is enabled for page sizes indicated by that mask value when page table entry bit 63 is set.

If Svnapot is present, "Svnapot_page_mask" is a mask of page sizes for which contiguous pages can be created. For example, a value of 0x10000 implies that 64KiB contiguous pages are supported.

1.10.9 Svpbmt

Parameter "Svpbmt" is 0 on this variant, meaning that page-based memory types are not implemented. if "Svpbmt" is set to 1 then page-based memory types are indicated by page table entry bits 62:61.

Note that except for their effect on Page Faults, the encoded memory types do not alter the behavior of this model, which always implements strongly-ordered non-cacheable semantics.

1.10.10 Ssnpm

Parameter "Ssnpm" is "none" on this variant, meaning Supervisor next level pointer masking not implemented.

1.10.11 Smnpm

Parameter "Smnpm" is "none" on this variant, meaning Machine next level pointer masking not implemented.

1.10.12 Smmpm

Parameter “Smmpm” is “none” on this variant, meaning Machine pointer masking not implemented.

1.10.13 Svinval

Parameter “Svinval” is 0 on this variant, meaning that fine-grained address-translation cache invalidation instructions are not implemented. if “Svinval” is set to 1 then fine-grained address-translation cache invalidation instructions sinval.vma, sfence.w.inval and sfence.inval.ir are implemented.

1.10.14 Svadu

Parameter “Svadu” is 0 on this variant, meaning that xenvcfg control of hardware update of PTE A and D bits is not implemented. if “Svadu” is set to 1 then xenvcfg control of hardware update of PTE A and D bits is implemented.

1.10.15 Ssdbltrp

Parameter “Ssdbltrp” is 0 on this variant, meaning that S-mode double trap extensions are not implemented. if “Ssdbltrp” is set to 1 then S-mode double trap extensions are implemented.

1.10.16 Smdbltrp

Parameter “Smdbltrp” is 0 on this variant, meaning that M-mode double trap extensions are not implemented. if “Smdbltrp” is set to 1 then M-mode double trap extensions are implemented.

1.10.17 Smstateen

Parameter “Smstateen” is 0 on this variant, meaning that machine/supervisor/hypervisor state enable CSRs are undefined. if “Smstateen” is set to 1 then machine/supervisor/hypervisor state enable CSRs are defined.

Set Ssstateen to True and Smstateen to False to implement only the supervisor/hypervisor state enable CSRs.

1.10.18 Sstc

Parameter “Sstc” is 0 on this variant, meaning that stimecmp is not implemented. if “Sstc” is set to 1 then stimecmp is implemented.

1.10.19 Sscofpmf

Parameter "Sscofpmf" is 0 on this variant, meaning that count overflow and mode-based filtering extension CSRs are undefined. if "Sscofpmf" is set to 1 then count overflow and mode-based filtering extension CSRs are defined.

Note that this model implements CSR state only for this extension; hardware performance counters themselves are implementation-specific and not implemented.

1.10.20 Smcntrpmf

Parameter "Smcntrpmf" is 0 on this variant, meaning that cycle and instret mode-based filtering extension CSRs are undefined. if "Smcntrpmf" is set to 1 then cycle and instret mode-based filtering extension CSRs are defined.

1.10.21 Smcsrind

Parameter "Smcsrind" is 0 on this variant, meaning that indirect CSR access is not implemented. if "Smcsrind" is set to 1 then indirect CSR access is implemented. If Supervisor mode is implemented, this also implicitly enables Sscsrind.

1.10.22 Smcdeleg

Parameter "Smcdeleg" is 0 on this variant, meaning that supervisor counter delegation is not implemented. if "Smcdeleg" is set to 1 then supervisor counter delegation is implemented. This also implicitly enables Ssccfg.

1.10.23 Ssqosid

Parameter "Ssqosid" is 0 on this variant, meaning that CSR srmcfg is undefined. if "Ssqosid" is set to 1 then CSR srmcfg is defined.

1.10.24 Zawrs

Parameter "Zawrs" is 0 on this variant, meaning that wait-for-reservation-set instructions are not implemented. if "Zawrs" is set to 1 then wait-for-reservation-set instructions are implemented, in which case parameter "TW_time_limit" is used to specify the nominal cycle delay for wrs.nto, and parameter "STO_time_limit" is used to specify the nominal cycle delay for wrs.sto.

1.10.25 Zimop

Parameter "Zimop" is 0 on this variant, meaning that maybe operations are not implemented. if "Zimop" is set to 1 then maybe operations are implemented.

1.10.26 Zcmop

Parameter "Zcmop" is 0 on this variant, meaning that compressed maybe operations are not implemented. if "Zcmop" is set to 1 then compressed maybe operations are implemented.

1.10.27 Zibi

Parameter "Zibi" is 0 on this variant, meaning that branch-with-immediate instructions are not implemented. if "Zibi" is set to 1 then branch-with-immediate instructions are implemented.

1.11 CLIC

The model can be configured to implement a Core Local Interrupt Controller (CLIC) using parameter "CLICLEVELS"; when non-zero, the CLIC is present with the specified number of interrupt levels (2-256), as described in the RISC-V Core-Local Interrupt Controller specification, and further parameters are made available to configure other aspects of the CLIC. "CLICLEVELS" is zero in this variant, indicating that a CLIC is not implemented.

The model can be configured either to use an internal CLIC model (if parameter "externalCLIC" is False) or to present a net interface to allow the CLIC to be implemented externally in a platform component (if parameter "externalCLIC" is True). When the CLIC is implemented internally, net ports for standard interrupts and additional local interrupts are available. When the CLIC is implemented externally, a net port interface allowing the highest-priority pending interrupt to be delivered is instead present. This is described below.

1.11.1 CLIC Common Parameters

This section describes parameters applicable whether the CLIC is implemented internally or externally. These are:

"CLIC_version": this defines the version of the CLIC specification that is implemented. See the References section of this document for more information.

"CLICANDBASIC": this enum parameter indicates whether only the CLICinterrupt controller is present (if "NoBasic") or both the CLIC and basic (AKA "CLINT") interrupt controllers are present, with common interrupts shared between the two (if "BasicCommon") or separate interrupts (if "BasicSeparate"). The default value in this variant is NoBasic.

"local_int_num": This Uns32 parameter specifies the number of local interrupts implemented in the basic interrupt mode. These interrupts are numbered starting at 16, as the first 16 interrupts in the basic interrupt controller have architecturally defined meanings. The basic local interrupts correspond to bits 16 and above in the mie/mip CSRS. The maximum value for this parameter is 16 for RV32 and 48 for RV64. The default value in this variant is 0.

"NUM_INTERRUPT": This Uns32 specifies the number of interrupts in the CLIC. If the clicinfo CSR is present then the num_interrupt field will be set to this value. The maximum value for this parameter is 4096. The minimum value for this parameter is 2, although setting it to 0 will set it

to the local_int_num value, if that is greater than 2, for backwards compatibility. The default value in this variant is 0. Note that the clic_imp_list can also change the effective value used, by listing interrupt numbers above the specified value.

"CLICXNXTI": this Boolean parameter indicates whether xnxti CSRs are implemented (if True) or unimplemented (if False). The default value in this variant is False.

"CLICXCSW": this Boolean parameter indicates whether xscratchcsw and xscratchcswl CSRs registers are implemented (if True) or unimplemented (if False). The default value in this variant is False.

"tvt_undefined": this Boolean parameter indicates whether xtvt CSRs registers are implemented (if True) or unimplemented (if False). If the registers are unimplemented then the model will use basic mode vectored interrupt semantics based on the xtvec CSRs instead of Selective Hardware Vectoring semantics described in the specification. The default value in this variant is False.

"intthresh_undefined": this Boolean parameter indicates whether xintthresh CSRs are implemented (if True) or unimplemented (if False). The default value in this variant is False.

"INTTHRESHBITS": if xintthresh CSRs are implemented, this parameter indicates the number of implemented bits in the "th" field. The default value in this variant is 0.

"mclicbase_undefined": this Boolean parameter indicates whether the mclicbase CSR register is implemented (if True) or unimplemented (if False); for CLIC versions after 0.9-draft-20220315, this feature is not configurable and this parameter is always True. The default value in this variant is True.

1.11.2 CLIC Internal-Implementation Parameters

This section describes parameters applicable only when the CLIC is implemented internally. These are:

"mclicbase": this parameter specifies the CLIC base address for M-mode interrupts in physical memory. The default value in this variant is 0x0.

"sclicbase": this parameter specifies the CLIC base address for S-mode interrupts in physical memory. The default value in this variant is 0x0.

"CLICCFGMBITS": this Uns32 parameter indicates the maximum number of bits that can be specified as implemented in cliccfg.nmbits, and also indirectly defines CLICPRIVMODES. For cores which implement only Machine mode, or which implement only Machine and User modes but not the N extension, the parameter is absent ("CLICCFGMBITS" must be zero in these cases). The default value in this variant is 0.

"CLICCFGLBITS": this Uns32 parameter indicates the maximum number of bits that can be specified as implemented in cliccfg.nlbits. The default value in this variant is 0. See also parameter "nlbits_valid" which allows fine-grain specification of legal values in cliccfg.nlbits.

"nlbits_valid": this Uns32 parameter is a bitmask of legal values that can be specified as implemented in the WARL cliccfg.nlbits field. As an example, a value of 8 in cliccfg.nlbits is legal only if bit 8 is set in nlbits_valid. The default value of "nlbits_valid" in this variant is 0x0.

"CLICSELHVEC": this Boolean parameter indicates whether Selective Hardware Vectoring is supported (if True) or unsupported (if False). The default value in this variant is False.

"forceCLICmmio": this Boolean parameter indicates whether the MMIO interface is present when CLIC_version is set to version 0.10-draft-20250317 or later (it is ignored for earlier versions.) As of CLIC version 0.10 the indirect CSR method is required, but the MMIO may optionally be implemented also, by setting this parameter to True. The default value in this variant is False.

"force_clicinfo": this Boolean parameter forces the clicinfo register to be present. This has no effect when the CLIC_version selection is earlier than 0.9.20221108, where clicinfo is always prsent. The default value in this variant is False.

1.11.3 CLIC Individual Interrupt Configuration Parameters

The following behaviors are individually configurable for CLIC interrupts using the set of parameters described in this section:

- whether the interrupt is implemented or not. Registers for unimplemented interrupts are read-only zeros, writes ignored, and no input port is created for them.

- whether the clicintattr.MODE field is read-write or read-only, and whether it is initialized to Machine (3) or Supervisor (1).

- whether the clicintattr.TRIG field is read-write or read-only, and whether it is initialized to POSLEVEL (0), POSEDGE (1), NEGLEVEL (2) or NEGEDGE (3).

The following two "clic_X_default" parameters are used to define the default behavior of the clicintattr MODE and TRIG fields. This will be the behavior for any interrupt not included in any of the "clic_X_list" parameters described later.

"clic_mode_default": this enum parameter specifies the default behavior for the clicintattr.MODE field for all implemented CLIC interrupts. The supported values are "RW_Machine", "RW_Supervisor", "RO_Machine" or "RO_Supervisor". The default value in this variant is RW_Machine.

"clic_trig_default": this enum parameter specifies the default behavior for the clicintattr.TRIG field for all implemented CLIC interrupts. The supported values are "RW_" or "RO_", followed by "POSLEVEL", "POSEDGE", "NEGLEVEL" or "NEGEDGE". The default value in this variant is RW_POSLEVEL.

The following parameters are all comma-separated lists of interrupt numbers (they may be decimal numbers or hex numbers preceded by "0x"). White space is permitted. The largest number allowed is 4095, the highest interrupt number supported by the CLIC architecture. It is a fatal error to specify an invalid list.

The behavior is undefined when the same interrupt number is specified in multiple lists that configure conflicting behavior.

"clic_imp_list" and "clic_unimp_list": interrupts 0..NUM_INTERRUPTS-1 are implemented by default, and these list parameters are used to refine that default configuration further. Interrupt numbers listed in "clic_imp_list" will be implemented, while those in "clic_unimp_list" will not be implemented. The default values in this variant are clic_imp_list="" and clic_unimp_list="".

"clic_mode_RO_list", "clic_mode_RW_list", "clic_mode_M_list" and "clic_mode_S_list": These are used to override the default behavior of the clicintattr.MODE field, both whether the field is writable and the initial value for the field. The default values in this variant are clic_mode_RO_list="", clic_mode_RW_list="", clic_mode_M_list="", and clic_mode_S_list="".

"clic_trig_RO_list", "clic_trig_RW_list", "clic_trig_posedge_list" and "clic_trig_negedge_list", "clic_trig_poslevel_list" and "clic_trig_neglevel_list": These are used to override the default behavior of the clicintattr.TRIG field, both whether the field is writable and the initial value for the field. The default values in this variant are clic_trig_RO_list="", clic_trig_RW_list="", clic_trig_posedge_list="", clic_trig_negedge_list="", clic_trig_poslevel_list="", and clic_trig_neglevel_list="".

1.11.4 CLIC Deprecated Parameters

The following parameters are deprecated in favor of the more general-purpose "clic_X_list" parameters described above. Everything that is configurable by the deprecated parameters can and should instead be done using those updated parameters.

"posedge_0_63" (deprecated): this Uns64 parameter specifies a mask for CLIC interrupts 0 to 63 indicating whether an interrupt is fixed positive-edge triggered. If the corresponding mask bit for interrupt N is 1 then the clicintattr.trig field for that interrupt is read-only and initialized appropriately. If zero, then the trig field is either fixed level triggered (see below) or is writable. The default value in this variant is posedge_0_63=0x0.

"posedge_other" (deprecated): this Boolean parameter indicates whether CLIC interrupts 64 and above are fixed positive-edge triggered. If True then the clicintattr.trig fields for these interrupts are read-only and initialized appropriately. If False, then the trig fields are either fixed level triggered (see below) or are writable. The default value in this variant is posedge_other=False.

"poslevel_0_63" (deprecated): this Uns64 parameter specifies a mask for CLIC interrupts 0 to 63 indicating whether an interrupt is fixed positive-level triggered. If the corresponding mask bit for interrupt N is 1 then the clicintattr.trig field for that interrupt is read-only and initialized appropriately. If zero, then the trig field is writable. This is ignored when the interrupt is configured as fixed positive-edge triggered (see above). The default value in this variant is poslevel_0_63=0x0.

"poslevel_other" (deprecated): this Boolean parameter indicates whether CLIC interrupts 64 and above are fixed positive level triggered. If True then the clicintattr.trig fields for these interrupts are read-only and initialized appropriately. If False, then the trig fields are writable. This is ignored when the other interrupts are configured as fixed positive-edge triggered (see above). The default value in this variant is poslevel_other=False.

"CSIP_present" (deprecated): this Boolean parameter indicates whether the CSIP interrupt is implemented (if True) or unimplemented (if False). If implemented, the interrupt is positive-edge triggered, and has id 12 or 16, depending on the CLIC version selected. The default value in this variant is False.

1.11.5 CLIC External-Implementation Net Port Interface

When the CLIC is externally implemented, net ports are present allowing the external CLIC model to supply the highest-priority pending interrupt and to be notified when interrupts are handled. These are:

"irq_id_i": this input should be written with the id of the highest-priority pending interrupt.

"irq_lev_i": this input should be written with the highest-priority interrupt level.

"irq_sec_i": this 2-bit input should be written with the highest-priority interrupt security state (00:User, 01:Supervisor, 11:Machine).

"irq_shv_i": this input port should be written to indicate whether the highest-priority interrupt should be direct (0) or vectored (1). If the "tvt_undefined parameter" is False, vectored interrupts will use selective hardware vectoring, as described in the CLIC specification. If "tvt_undefined" is True, vectored interrupts will behave like basic mode vectored interrupts.

"irq_i": this input should be written with 1 to indicate that the external CLIC is presenting an interrupt, or 0 if no interrupt is being presented.

"irq_ack_o": this output is written by the model on entry to the interrupt handler (i.e. when the interrupt is taken). It will be written as an instantaneous pulse (i.e. written to 1, then immediately 0).

"irq_id_o": this output is written by the model with the id of the interrupt currently being handled. It is valid during the instantaneous irq_ack_o pulse.

"sec_lvl_o": this output signal indicates the current secure status of the processor, as a 2-bit value (00=User, 01:Supervisor, 11=Machine).

1.11.6 CLIC Versions

The CLIC specification has been under active development. To enable simulation of hardware that may be based on an older version of the specification, the model implements behavior for a number of previous versions of the specification. The differing features of these are listed below, in chronological order.

1.11.7 Version 20180831

Legacy version of August 31 2018, compatible with early SiFive cores.

1.11.8 Version 0.9-draft-20191208

Stable 0.9 version of December 8 2019.

1.11.9 Version 0.9-draft-20220315

Stable 0.9 version of March 15 2022, with these changes compared to version 0.9-draft-20191208:

- level-sensitive interrupts may no longer be set pending by software;

- if there is an access exception on table load when Selective Hardware Vectoring is enabled, both xtval and xepc hold the faulting address;

- if the xinhv bit is set, the target address for an xret instruction is obtained by a load from address xepc.

1.11.10 Version 0.9-draft-20221108

Stable 0.9 version of November 8 2022, with these changes compared to version 0.9-draft-20191208:

- vector table reads now require execute permission (not read permission);

- CSIP now has id 16 and is treated as the first local interrupt (previously, it had id 12 and was not considered to be a local interrupt);

- algorithm used for xscratchcsw accesses changed;

- new parameter INTTHRESHBITS allows implemented bits in xintthresh CSRs to be restricted.

- whether interrupt handling is in CLIC or CLINT mode is now determined by mtvec.MODE at all privilege levels (when both modes are supported).

- xintstatus registers have been relocated at read-only CSR addresses (e.g. 0xF46 for mintstatus).

- cliccfg.nvbits field has been removed.

- clicinfo memory-mapped register has been removed.

1.11.11 Version 0.9-draft-20230801

Stable 0.9 version of August 1 2023, with these changes compared to version 0.9-draft-20221108:

- xintstatus registers have been relocated at different read-only CSR addresses (e.g. 0xFB1 for mintstatus);

- 32-bit xcliccfg registers are present on a configuration page preceding interrupt status pages, containing separate unlbits, snlbits and mnlbits fields for each mode;

- if the hart is currently running at privilege mode x, an mret, sret, dret or mnret instruction that changes privilege mode sets xintthresh=0;

- vector table reads now require execute permission (not read permission);

- xepc is forced to table-entry alignment on xRET when inhv is active;

- inhv is now held in exception mode, not interrupt mode.

1.11.12 Version 0.10-draft-20250317

Draft version of March 17 2025, with these changes compared to version 0.9-draft-20230801:

- Indirect CSRs replace the CLIC Memory-Mapped Registers. (new parameter “forceCLICmmio” added to implement MMIO registers in addition to indirect CSRs.)

- CSRs and parameters associated with suclic extension are no longer present. The proposed User mode interrupt N extension is no longer allowed to be enabled when a CLIC version 0.10 or later is present.

- Add mandatory reset to 0 of mcause.mpil.

- CLIC is allocated bit 53 in the stateen0 register to block access to the ssclic CSRs.

1.11.13 Version master

Unstable master version as of 17 March 2025 (commit 62092fc)

1.12 Advanced Interrupt Architecture

The model can be configured to implement the Advanced Interrupt Architecture (AIA) interface using Boolean parameter “Smaia”; when True, the AIA interface is present as described in the RISC-V Advanced Interrupt Architecture specification, and further parameters are made available to configure other aspects of the interface. “Smaia” is False in this variant, indicating that the AIA interface is not implemented.

1.13 WorldGuard Extension

The model can be configured to implement the WorldGuard ISA extension using parameter “WG_version”; when not “none”, the WorldGuard ISA extension is present as described in the RISC-V WorldGuard specification, and further parameters are made available to configure other aspects of the extension. “WG_version” is “none” in this variant, indicating that WorldGuard ISA extension is not implemented.

1.14 Zalrsc extension (Load-Reserved/Store-Conditional Locking)

By default, LR/SC locking is implemented automatically by the model and simulator, with a reservation granule defined by the “lr_sc_grain” parameter. Setting “lr_sc_grain” to 1 means the reservation granule is the access size of the lr instruction making the reservation, otherwise the reservation granule is always equal to the “lr_sc_grain” in bytes. Note that any value less than 4 for RV32 or 8 for RV64 will be mapped to 1, and any value not a power of 2 will be mapped to the next lowest power of 2.

In this variant, “lr_sc_grain” is 1, meaning the reservation granule is the access size of the lr instruction making the reservation.

Parameter “lr_sc_match_size” is used to specify whether data sizes of lr and sc instructions must match for the sc instruction to succeed. In this variant, “lr_sc_match_size” is False.

Note that when “lr_sc_grain” is 1, it is never possible for an sc.d instruction to succeed after a preceding lr.w instruction because the reservation granule does not include all data bytes to be written by the sc.d instruction. However, an sc.w instruction may match a preceding lr.d instruction if parameter “lr_sc_match_size” is False.

Parameter “amo_aborts_lr_sc” is used to specify whether AMO operations abort any active LR/SC pair. In this variant, “amo_aborts_lr_sc” is False.

Parameter “trap_preserves_lr” is used to specify whether a trap preserves active LR/SC state. In this variant, “trap_preserves_lr” is False.

Parameter “xret_preserves_lr” is used to specify whether an xret instruction preserves active LR/SC state. In this variant, “xret_preserves_lr” is False.

Parameter “lr_sc_use_pa” is used to specify whether LR/SC uses physical addresses, so reservations are supported across virtual aliases. In this variant, “lr_sc_use_pa” is False.

Parameter “lr_sc_abort_self” is used to specify whether LR/SC can be aborted by a store on the same hart that executed the LR instruction In this variant. In this variant, “lr_sc_abort_self” is False.

It is possible to implement locking externally to the model in a platform component, using the “LR_address”, “SC_address” and “SC_valid” net ports, as described below.

The “LR_address” output net port is written by the model with the address used by a load-reserved instruction as it executes. This port should be connected as an input to the external lock management component, which should record the address, and also that an LR/SC transaction is active.

The “SC_address” output net port is written by the model with the address used by a store-conditional instruction as it executes. This should be connected as an input to the external lock management component, which should compare the address with the previously-recorded load-reserved address, and determine from this (and other implementation-specific constraints) whether the store should succeed. It should then immediately write the Boolean success/fail code to the “SC_valid” input net port of the model. Finally, it should update state to indicate that an LR/SC transaction is no longer active.

It is also possible to write zero to the “SC_valid” input net port at any time outside the context of a store-conditional instruction, which will mark any active LR/SC transaction as invalid.

Parameter “Zacas” is used to specify whether atomic compare-and-swap instructions are implemented. In this variant, “Zacas” is False.

1.15 Active Atomic Operation Indication

The “AMO_active” output net port is written by the model with a code indicating any current atomic memory operation while the instruction is active. The written codes are:

0: no atomic instruction active

1: AMOMIN active

2: AMOMAX active

3: AMOMINU active

4: AMOMAXU active

5: AMOADD active

6: AMOXOR active

7: AMOOR active

8: AMOAND active

9: AMOSWAP active

10: AMOCAS.W active (Zacas extension)

11: AMOCAS.D active (Zacas extension)

12: AMOCAS.Q active (Zacas extension)

13: LR active

14: SC active

1.16 Interrupts

The "reset" port is an active-high reset input. The processor is halted when "reset" goes high and resumes execution from the reset address specified using the "reset_address" parameter or "reset_addr" port when the signal goes low. The "mcause" register is cleared to zero.

The "nmi" port is an active-high NMI input. The processor resumes execution from the address specified using the "nmi_address" parameter or "nmi_addr" port when the NMI signal goes high. The "mcause" register is cleared to zero.

All other interrupt ports are active high. For each implemented privileged execution level, there are by default input ports for software interrupt, timer interrupt and external interrupt; for example, for Machine mode, these are called "MSWInterrupt", "MTimerInterrupt" and "MExternalInterrupt", respectively. When the N extension is implemented, ports are also present for User mode. Parameter "unimp_int_mask" allows the default behavior to be changed to exclude certain interrupt ports. The parameter value is a mask in the same format as the "mip" CSR; any interrupt corresponding to a non-zero bit in this mask will be removed from the processor and read as zero in "mip", "mie" and "mideleg" CSRs (and Supervisor and User mode equivalents if implemented).

Parameter "posedge_sswi" is used to specify that the SSWInterrupt is edge triggered. This must be set to True if this input is driven by an ACLINT, due to the unique behavior of its SETSSIP register. In this variant, "posedge_sswi" is False.

Parameter "external_int_id" can be used to enable extra interrupt ID input ports on each hart. If the parameter is True then when an external interrupt is taken the value on the ID port is sampled and used to fill the Exception Code field in the relevant "xcause" CSR. For Machine External

interrupts, the extra interrupt ID port is called "MExternalInterruptID"; for Supervisor External interrupts, the extra interrupt ID port is called "SExternalInterruptID".

The "deferint" port is an active-high artifact input that, when written to 1, prevents any pending-and-enabled interrupt being taken (normally, such an interrupt would be taken on the next instruction after it becomes pending-and-enabled). The purpose of this signal is to enable alignment with hardware models in step-and-compare usage.

1.17 Triggering Exceptions Externally

The following artifact input net port(s) are provided to support triggering exceptions externally (e.g. by a DV test harness or an external bus controller model.)

illegalinstruction - This port is used to immediately trigger an Illegal instruction exception (exception code 2). The exception is triggered when a leading edge (0->1 transition) is seen on the net. If the processor is in a WFI state with a bounded time limit enabled, then the illegal instruction exception follows the conventions for an event causing the hart to resume from WFI mode, namely the EPC will be set to the instruction following the WFI instruction when taking the exception.

hardwareerror - This port is used to immediately trigger a Hardware error exception (exception code 19). The exception is triggered when a leading edge (0->1 transition) is seen on the net. The tval for the exception may be set by driving the artifactdata port (see below) with the desired value immediately before the trigger.

artifactdata - This port is used to provide additional data to be used when another artifact port triggers. When a port that uses artifact data triggers, the most recent value written to the port is sampled, and the value on the port is reset to 0 so a subsequent artifact port trigger will not use a stale value. Do not rely on the previous value being used by subsequent triggers unless it is written again (or it is 0.)

1.18 Debug Mode (Sdext)

The model can be configured to implement Debug mode using parameter "debug_mode". This implements features described in Chapter 4 of the RISC-V External Debug Support specification with version specified by parameter "debug_version" (see References). Some aspects of this mode are not defined in the specification because they are implementation-specific; the model provides infrastructure to allow implementation of a Debug Module using a custom harness. Features added are described below.

Parameter "debug_mode" can be used to specify four different behaviors, as follows:

1. If set to value "vector", then operations that would cause entry to Debug mode result in the processor jumping to the address specified by the "debug_address" parameter. It will execute at this address, in Debug mode, until a "dret" instruction causes return to non-Debug mode. Any exception generated during this execution will cause a jump to the address specified by the "dexc_address" parameter.

2. If set to value "interrupt", then operations that would cause entry to Debug mode result

in the processor simulation call (e.g. opProcessorSimulate) returning, with a stop reason of OP_SR_INTERRUPT. In this usage scenario, the Debug Module is implemented in the simulation harness.

3. If set to value "halt", then operations that would cause entry to Debug mode result in the processor halting. Depending on the simulation environment, this might cause a return from the simulation call with a stop reason of OP_SR_HALT, or debug mode might be implemented by another platform component which then restarts the debugged processor again.

4. If set to value "inject", then operations that would cause entry to Debug mode result in the processor continuing to execute from the current address in Debug mode. The harness should detect that Debug mode has been entered by monitoring the "DM" integration support register, and inject Debug-mode instructions one at a time using function opProcessorSimulateInstruction. Debug mode is exited by either an explicit write of False to the "DM" register or by execution of an injected "dret" instruction, as described by "Debug State Exit" below.

Note that parameter "debug_mode" can be also be set to "none" to specify that Sdext is not implemented.

1.18.1 Debug State Entry

The specification does not define how Debug mode is implemented. In this model, Debug mode is enabled by a Boolean pseudo-register, "DM". When "DM" is True, the processor is in Debug mode. When "DM" is False, mode is defined by "mstatus" in the usual way.

Entry to Debug mode can be performed in any of these ways:

1. By writing True to register "DM" (e.g. using opProcessorRegWrite) followed by simulation of at least one cycle (e.g. using opProcessorSimulate) - in this case, dcsr.cause will report a cause of trigger (2);

2. By writing a 1 then 0 to net "haltreq" (using opNetWrite) followed by simulation of at least one cycle (e.g. using opProcessorSimulate) - in this case, dcsr.cause will report a cause of haltreq (3);

3. By writing a 1 to net "resethaltreq" (using opNetWrite) while the "reset" signal undergoes a negedge transition, followed by simulation of at least one cycle (e.g. using opProcessorSimulate) - in this case, dcsr.cause will report a cause of resethaltreq (5) or haltreq (3), depending on the value of parameter "no_resethaltreq";

4. By executing an "ebreak" instruction when Debug mode entry for the current processor mode is enabled by dcsr.ebreakm, dcsr.ebreaks or dcsr.ebreaku - in this case, dcsr.cause will report a cause of ebreak (1);

5. By executing a single instruction when Debug mode entry for the current processor mode is enabled by dcsr.step - in this case, dcsr.cause will report a cause of step (4);

6. By a Trigger Module trigger, when that trigger is configured to enter Debug mode - in this case, dcsr.cause will report a cause of trigger (2).

In all cases, the processor will save required state in "dpc" and "dcsr" and then perform actions described above, depending in the value of the "debug_mode" parameter.

1.18.2 Debug State Exit

Exit from Debug mode can be performed in either of these ways:

1. By writing False to register "DM" (e.g. using opProcessorRegWrite) followed by simulation of at least one cycle (e.g. using opProcessorSimulate);

2. By executing an "dret" instruction when Debug mode.

In both cases, the processor will perform the steps described in section 4.6 (Resume) of the Debug specification.

1.18.3 Debug Registers

When Debug mode is enabled, registers "dcsr" and "dpc" are implemented as described in the specification. Registers "dscratch0" and "dscratch1" may also be implemented (see parameters below). Implemented registers may be manipulated externally by a Debug Module using opProcessorRegRead or opProcessorRegWrite; for example, the Debug Module could write "dcsr" to enable "ebreak" instruction behavior as described above, or read and write "dpc" to emulate stepping over an "ebreak" instruction prior to resumption from Debug mode.

Parameter "dscratch0_undefined" is used to specify whether the "dscratch0" CSR is undefined, in which case accesses to it trap to Machine mode. In this variant, "dscratch0_undefined" is 0.

Parameter "dscratch1_undefined" is used to specify whether the "dscratch1" CSR is undefined, in which case accesses to it trap to Machine mode. In this variant, "dscratch1_undefined" is 0.

Parameter dcsr_stepie is used to specify the behavior of the dcsr.stepie field. This is currently configured as read/write, reset to 0.

Parameter dcsr_stopcount is used to specify the behavior of the dcsr.stopcount field. This is currently configured as read/write, reset to 0.

Parameter dcsr_stoptime is used to specify the behavior of the dcsr.stoptime field. This is currently configured as read/write, reset to 0.

Parameter dcsr_mprven is used to specify the behavior of the dcsr.mprven field. This is currently configured as read/write, reset to 0.

1.18.4 Debug Mode Execution

The specification allows execution of code fragments in Debug mode. A Debug Module implementation can cause execution in Debug mode by the following steps:

1. Write the address of a Program Buffer to the program counter using opProcessorPCSet;

2. If "debug_mode" is set to "halt", write 0 to pseudo-register "DMStall" (to leave halted state);

3. If entry to Debug mode was handled by exiting the simulation callback, call opProcessorSimulate or opRootModuleSimulate to resume simulation.

Debug mode will be re-entered in these cases:

1. By execution of an “ebreak” instruction; or:

2. By execution of an instruction that causes an exception.

In both cases, the processor will either jump to the debug exception address, or return control immediately to the harness, with stopReason of OP_SR_INTERRUPT, or perform a halt, depending on the value of the “debug_mode” parameter.

1.18.5 Debug Single Step

When in Debug mode, the processor or harness can cause a single instruction to be executed on return from that mode by setting dcsr.step. After one non-Debug-mode instruction has been executed, control will be returned to the harness. The processor will remain in single-step mode until dcsr.step is cleared.

1.18.6 Debug Event Priorities

The model supports three different models for determining which debug exception occurs when step, execute address, resethaltreq and haltreq events are all pending. These options are listed below, with highest-priority event first:

1. when parameter “debug_priority”=“sxh”: step ->execute address ->resethaltreq ->haltreq;

2. when parameter “debug_priority”=“shx”: step ->resethaltreq ->haltreq ->execute address;

3. when parameter “debug_priority”=“hsx”: resethaltreq ->haltreq ->step ->execute address.

1.18.7 Debug Ports

Port “DM” is an output signal that indicates whether the processor is in Debug mode

Port “haltreq” is a rising-edge-triggered signal that triggers entry to Debug mode (see above).

Port “resethaltreq” is a level-sensitive signal that triggers entry to Debug mode after reset (see above).

1.18.8 Debug Mode Versions

Debug mode specification has been under active development. To enable simulation of hardware that may be based on an older version of the specification, the model implements behavior for a number of versions of the specification. The differing features of these are listed below, in chronological order.

1.18.9 Version 0.13.2-DRAFT

0.13.2-DRAFT version of March 22 2019.

1.18.10 Version 0.14.0-DRAFT

0.14.0-DRAFT version of November 6 2020.

1.18.11 Version 1.0.0-STABLE

1.0.0-STABLE version of February 9 2022.

1.18.12 Version 1.0-STABLE

1.0-STABLE version of December 28 2022, with these changes compared to version 1.0.0-STABLE:

- nmi is moved from etrigger to itrigger and is now subject to the mode bits in that trigger.

1.19 Trigger Module (Sdtrig)

This model is configured with a trigger module, implementing a subset of the behavior described in Chapter 5 of the RISC-V External Debug Support specification with version specified by parameter "debug_version" (see References).

1.19.1 Trigger Module Restrictions

The model currently supports tdata1 of type 0, type 2 (mcontrol), type 3 (icount), type 4 (itrigger), type 5 (etrigger) and type 6 (mcontrol6). icount triggers are implemented for a single instruction only, with count hard-wired to 1 and automatic zeroing of mode bits when the trigger fires.

1.19.2 Trigger Module Parameters

Parameter "trigger_num" is used to specify the number of implemented triggers. In this variant, "trigger_num" is 4. Set this parameter to 0 to specify that Sdtrig is not implemented.

Parameter "tinfo" is used to specify the default value of the read-only "tinfo" registers, which indicates the trigger types supported and also version information which controls the behavior of "mcontrol6". In this variant, "tinfo" is 0x100807c.

In addition, parameters "tinfo0" through "tinfo15" may be used to specify non-default values for individual tinfo registers. See the "Parameters values and limits" section for info on the default setting for each tinfo register.

Parameter "trigger_match" is used to specify the legal "match" values for triggers of types 2 and 6. This parameter is a bitmask with 1 bits corresponding to legal values; for example, a "trigger_match" of 0xd, means that matches of types 0, 2 and 3 are supported. In this variant, "trigger_match" is 0x333f.

Parameter "tdata2_undefined" is used to specify whether the "tdata2" register is undefined, in which case reads of it trap to Machine mode. In this variant, "tdata2_undefined" is 0.

Parameter “tdata3_undefined” is used to specify whether the “tdata3” register is undefined, in which case reads of it trap to Machine mode. In this variant, “tdata3_undefined” is 0.

Parameter “tinfo_undefined” is used to specify whether the “tinfo” register is undefined, in which case reads of it trap to Machine mode. In this variant, “tinfo_undefined” is 0.

Parameter “tcontrol_undefined” is used to specify whether the “tcontrol” register is undefined, in which case accesses to it trap to Machine mode. In this variant, “tcontrol_undefined” is 0.

Parameter “mcontext_undefined” is used to specify whether the “mcontext” register is undefined, in which case accesses to it trap to Machine mode. In this variant, “mcontext_undefined” is 0.

Parameter “scontext_undefined” is used to specify whether the “scontext” register is undefined, in which case accesses to it trap to Machine mode. In this variant, “scontext_undefined” is 0.

Parameter “mscontext_undefined” is used to specify whether the “mscontext” register is undefined, in which case accesses to it trap to Machine mode. In this variant, “mscontext_undefined” is 0.

Parameter “amo_trigger” is used to specify whether load/store triggers are activated for AMO instructions. In this variant, “amo_trigger” is 0.

Parameter “no_hit” is used to specify whether the “hit” bits in tdata1 are unimplemented. In this variant, “no_hit” is 0.

Parameter “no_sselect_2” is used to specify whether the “sselect” field in “textra32”/“textra64” registers is unable to hold value 2 (indicating match by ASID is not allowed). In this variant, “no_sselect_2” is 0.

Parameter “mcontext_bits” is used to specify the number of writable bits in the “mcontext” register. In this variant, “mcontext_bits” is 13.

Parameter “scontext_bits” is used to specify the number of writable bits in the “scontext” register. In this variant, “scontext_bits” is 34.

Parameter “mvalue_bits” is used to specify the number of writable bits in the “mvalue” field in “textra32”/“textra64” registers; if zero, the “mselect” field is tied to zero. In this variant, “mvalue_bits” is 13.

Parameter “svalue_bits” is used to specify the number of writable bits in the “svalue” field in “textra32”/“textra64” registers; if zero, the “sselect” is tied to zero. In this variant, “svalue_bits” is 34.

Parameter “mcontrol_maskmax” is used to specify the value of field “maskmax” in the “mcontrol” register. In this variant, “mcontrol_maskmax” is 63.

Parameter “mcontrol_addl_comp” is used to specify whether the compare values for trigger types 2 (mcontrol) and 6 (mcontrol6) when select is 0 optionally includes ALL virtual addresses accessed, in addition to the required lowest VA. In this variant, “mcontrol_addl_comp” is False.

1.20 Debug Mask

It is possible to enable model debug messages in various categories. This can be done statically using the “debugflags” parameter, or dynamically using the “debugflags” command. Enabled messages

are specified using a bitmask value, as follows:

Value 0x002: enable debugging of PMP and virtual memory state;

Value 0x004: enable debugging of interrupt state;

Value 0x008: enable TLB consistency checking (automatically detect inconsistencies between cached TLB entries and the page table in memory, if these would affect model behavior).

All other bits in the debug bitmask are reserved and must not be set to non-zero values.

1.21 Integration Support

This model implements a number of non-architectural pseudo-registers, commands, and other features to facilitate integration.

1.21.1 Command "setPMA -attributes <attrs>-lo <addr>-hi <addr>"

This command allows PMA attributes to be set for the address range lo:hi. The required attributes are described by the "attrs" string, which can contain any combination of these characters:

"r": allow read access

"w": allow write access

"x": allow execute access

"a": disallow unaligned accesses

"A": disallow RVMC_USER1 accesses (often AMO and LR/SC)

"P": disallow RVMC_USER2 accesses (often push/pop)

"1": allow 1-byte accesses (if none of 1248 are specified then all are allowed)

"2": allow 2-byte accesses

"4": allow 4-byte accesses

"8": allow 8-byte accesses

<space>, "-": ignored, use for formatting

The command may be used multiple times, in which case PMA attributes for later commands override those specified for earlier ones where ranges overlap. A common idiom is to deny all access to the entire memory range in the first command before adding back permissions for subregions with subsequent commands.

1.21.2 Command "addLRSCRegion -grainSizeM1 <size-1>-lo <addr>-hi <addr>"

This command allows the LR/SC reservation grain size to be set for the address range lo:hi. For example, a value of 63 for the "grainSizeM1" argument means that the reservation grain size in the

indicated address range is 64 bytes. "grainSizeM1" must be one less than a power of two.

The command may be used multiple times, in which case LR/SC reservation grain sizes for later commands override those specified for earlier ones where ranges overlap.

1.21.3 Command "resetLRSCRegion -lo <addr>-hi <addr>"

This command allows the LR/SC reservation grain size to be reset for the address range lo:hi. When reset, the grain size for that region will be the default "lr_sc_grain" specified by the processor configuration.

1.21.4 Command "getCSRIndex -name <name>"

This command returns the index number of a named CSR, or -1 if that CSR does not exist.

1.21.5 Command "listCSRs"

This command lists all implemented CSRs in index order.

1.21.6 CSR Register External Implementation

If parameter "enable_CSR_bus" is True, an artifact 16-bit bus "CSR" is enabled. Slave callbacks installed on this bus can be used to implement modified CSR behavior (use opBusSlaveNew or icmMapExternalMemory, depending on the client API). A CSR with index 0xABC is mapped on the bus at address 0xABC0; as a concrete example, implementing CSR "time" (number 0xC01) externally requires installation of a read callback at address 0xC010 on the CSR bus.

If both read and write callbacks are installed, or if a read callback is installed and the CSR is in the read-only address space, then the read callback will be used to provide the value for both true accesses and for trace and API register read (using opRegRead, etc). However, if only a read callback is installed and the CSR is in the CSR read/write address space then the callback will be used for true register reads *only*; in this case, the *model* CSR implementation will be used for trace and API register read. This idiom allows values to be injected for volatile CSRs without changing fundamental model behavior.

An artifact net, "readcsr", can also be used to override the value apparently read from a CSR without resorting to the CSR bus. When a CSR is read into a GPR that is not "x0", this net is written with a value encoding the CSR number (in bits 11:0) and destination GPR number (in bits 20:16). To use this net:

1. Install a net monitor callback on "readcsr" using opNetWriteMonitorAdd;

2. When the callback is activated, extract the encoded CSR and GPR numbers;

3. If the CSR number corresponds to a CSR of interest, find the OP register corresponding to the GPR using opProcessorRegByIndex;

4. Use opProcessorRegWrite to modify the GPR value.

1.21.7 LR/SC Active Address

Artifact register "LRSCAddress" shows the active LR/SC lock address. The register holds all-ones if there is no LR/SC operation active or if LR/SC locking is implemented externally as described above. When parameter "lr_sc_match_size" is True and this is a 64-bit access, the least-significant bit of "LRSCAddress" is one (to indicate a 64-bit access). If parameter "lr_sc_match_size" is False or this is a 32-bit access, the least-significant bit of "LRSCAddress" is zero.

1.21.8 Page Table Walk Introspection

Artifact register "PTWStage" shows the active page table translation stage (0 if no stage active, 1 if HS-stage active, 2 if VS-stage active and 3 if G-stage active). This register is visibly non-zero only in a memory access callback triggered by a page table walk event.

Artifact register "PTWInputAddr" shows the input address of active page table translation. This register is visibly non-zero only in a memory access callback triggered by a page table walk event.

Artifact register "PTWLevel" shows the active level of page table translation (corresponding to index variable "i" in the algorithm described by Virtual Address Translation Process in the RISC-V Privileged Architecture specification). This register is visibly non-zero only in a memory access callback triggered by a page table walk event.

1.21.9 Page Table Walk Query

A banked set of registers provides information about the most recently completed page table walk. There are up to seven banks of registers: bank 0 is for stage 1 walks, bank 1 is for stage 2 walks, and banks 2-6 are for stage 2 walks initiated by stage 1 level 0-4 entry lookups, respectively. Banks 1-6 are present only for processors with the Virtualization Extension. The currently active bank can be set using register "PTWBankSelect". Register "PTWBankValid" is a bitmask indicating which banks contain valid data: for example, the value 0xb indicates that banks 0, 1 and 3 contain valid data.

Within each bank, there are registers that record addresses and values read during that page table walk. Register "PTWBase" records the table base address, register "PTWInput" contains the input address that starts a walk, register "PTWOutput" contains the result address and register "PTWPgSize" contains the page size ("PTWOutput" and "PTWPgSize" are valid only if the page table walk completes). Registers "PTWAddressL0"-"PTWAddressL4" record the addresses of level 0 to level 4 entries read, respectively. Register "PTWAddressValid" is a bitmask indicating which address registers contain valid data: bits 0-4 indicate "PTWAddressL0"-"PTWAddressL4", respectively, bit 5 indicates "PTWBase", bit 6 indicates "PTWInput", bit 7 indicates both "PTWOutput" and "PTWPgSize". For example, the value 0xe3 indicates that "PTWBase", "PTWInput", "PTWOutput", "PTWPgSize" and "PTWAddressL0" and "PTWAddressL1" are valid but "PTWAddressL2"-"PTWAddressL4" are not. Registers "PTWValueL0"-"PTWValueL4" contain page table entry values read at level 0 to level 4. Register "PTWValueValid" is a bitmask indicating which value registers contain valid data: bits 0-4 indicate "PTWValueL0"-"PTWValueL4", respectively.

1.21.10 Artifact Page Table Walks

Registers are also available to enable a simulation environment to initiate an artifact page table walk (for example, to determine the ultimate PA corresponding to a given VA). Register PTWUD initiates a U/HU-mode walk for a load/store address. Register PTWUX initiates a U/HU-mode walk for a fetch address. Registers PTWSD and PTWSX perform corresponding walks for S/HS mode accesses. If Hypervisor mode is implemented, registers PTWVUD and PTWVUX perform corresponding walks for VU mode accesses, and registers PTWVSD and PTWVSX perform corresponding walks for VS mode accesses. Each walk fills the query registers described above.

1.21.11 Page Table Walk Event

Event "pageTableWalk" triggers on completion of any page table walk (including artifact walks). An event callback registered on this event can access the Page Table Walk Query artifact registers described in the previous section.

1.21.12 Artifact Register "fflags_i"

If parameter "enable_fflags_i" is True, an 8-bit artifact register "fflags_i" is added to the model. This register shows the floating point flags set by the current instruction (unlike the standard "fflags" CSR, in which the flag bits are sticky).

1.21.13 External Stimulation of Illegal Instruction Trap

Artifact input net port "illegalinstr" allows Illegal Instruction traps to be raised externally. On a rising edge of the signal connected to this port, the hart will immediately take an Illegal Instruction trap with "xepc" set to the current program counter.

As a special case, if the hart is currently stalled by a WFI instruction ("wfi_is_nop" is False), it will be restarted and take either an Illegal Instruction or Virtual Instruction trap, based on the current processor mode and the governing TW bit.

1.21.14 Halt Reason Introspection

An artifact register HaltReason can be read to determine the reason or reasons that a processor is halted. This register is a bitfield, with the following encoding:

0x001: waiting in WFI

0x002: waiting in reset

0x004: waiting for debug

0x008: waiting for reservation set (wrs.*)

0x010: waiting for reservation set (wrs.sto)

0x020: waiting for any pending and enabled interrupt

0x040: waiting for custom reason with WFI semantics

0x080: waiting for custom reason with NMI semantics

0x100: waiting for custom reason (persists over reset)

0x200: waiting because of critical error (Smdbltrp)

An output net of the same name is also present. The net is written whenever the HaltReason changes.

1.22 Instruction Disassembly

This model implements a number of parameters to control instruction disassembly, as shown in trace output.

If parameter "use_hw_reg_names" is True, instruction disassembly shows hardware names x0-x31. If "use_hw_reg_names" is False, ABI names are shown instead.

If parameter "no_pseudo_inst" is True, instruction disassembly always shows true instructions. If "no_pseudo_inst" is False, pseudo-instructions are shown instead where applicable.

If parameter "show_c_prefix" is True, instruction disassembly of 16-bit instructions will include a compressed prefix (e.g. "c." or "cm."). If "show_c_prefix" is False, the compressed prefix will be omitted.

1.23 Semihosting

Extension libraries for semihosting of the host computer I/O and related facilities are available. A variety of protocols are supported as described below. These are all in the VLNV tree with vendor=riscv.ovpworld.org and library=semihosting.

By default, the pk semihosting (see below) is enabled. To select a different riscv.ovpworld.org semihost library, use the "semihostname" simulator parameter.

To disable semihosting entirely, for example when running an operating system where drivers are implemented along with I/O hardware emulated by platform peripherals, override the"defaultsemihost" processor parameter with the value True.

1.23.1 Proxy Kernel semihosting (default)

The Riscv Proxy Kernel semihosting Execution Environment Interface (EEI) is supported by the riscv.ovpworld.org semihosting library with name=pk. This uses the ecall or ebreak instruction with register a7 (on RVI) or x5 (on RVE) to hold the call type, while registers a0 thru a6 (on RVI) or x10 thru x15 (on RVE) hold argument values. See https://github.com/riscv-software-src/riscv-pk for details.

When using the Riscv GNU gcc toolchain this corresponds to using the command line option –specs=sim.specs (which is usually configured as the default behavior.)

1.23.2 Angel-compatible semihosting

An Angel-compatible semihosting Execution Environment Interface (EEI) is supported by the riscv.ovpworld.org semihosting library with name=semihost. This uses the ecall or ebreak instruction with register a0 to hold the call type, while register a1 contains a pointer to a data block with the argument values for that call type.

When using the Riscv GNU gcc toolchain this corresponds to using the command line option –specs=semihost.specs. See libgloss/riscv in https://sourceware.org/git/?p=newlib-cygwin.git;a=tree for details.

1.23.3 Newlib interception semihosting

Semihosting that intercepts newlib calls directly is supported by the riscv.ovpworld.org semihosting library with name=riscv32Newlib or riscv64Newlib. Using this semihost library removes any dependency on the EEI protocol implemented by the libgloss layer, but does not provide complete testing of the toolchain code or provide support for direct environment calls that do not go through newlib functions.

1.24 Limitations

Instruction pipelines are not modeled in any way. All instructions are assumed to complete immediately. This means that instruction barrier instructions (e.g. fence.i) are treated as NOPs, with the exception of any Illegal Instruction behavior, which is modeled.

Caches and write buffers are not modeled in any way. All loads, fetches and stores complete immediately and in order, and are fully synchronous. Data barrier instructions (e.g. fence) are treated as NOPs, with the exception of any Illegal Instruction behavior, which is modeled.

Real-world timing effects are not modeled: all instructions are assumed to complete in a single cycle.

Hardware Performance Monitor registers are not implemented and hardwired to zero.

The TLB is architecturally-accurate but not device accurate. This means that all TLB maintenance and address translation operations are fully implemented but the cache is larger than in the real device.

The internal CLIC does not model the optional interrupt triggers (clicinttrig registers.) These registers appear as hard-wired zeros.

1.25 Verification

All instructions have been extensively tested by Imperas, using tests generated specifically for this model and also reference tests from https://github.com/riscv/riscv-tests.

Also reference tests have been used from various sources including:

https://github.com/riscv/riscv-tests

https://github.com/ucb-bar/riscv-torture

The Imperas OVPsim RISC-V models are used in the RISC-V Foundation Compliance Framework as a functional Golden Reference:

https://github.com/riscv/riscv-compliance

where the simulated model is used to provide the reference signatures for compliance testing. The Imperas OVPsim RISC-V models are used as reference in both open source and commercial instruction stream test generators for hardware design verification, for example:

http://valtrix.in/sting from Valtrix

https://github.com/google/riscv-dv from Google

The Imperas OVPsim RISC-V models are also used by commercial and open source RISC-V Core RTL developers as a reference to ensure correct functionality of their IP.

1.26 References

The Model details are based upon the following specifications:

RISC-V Instruction Set Manual, Volume I: User-Level ISA (User Architecture Version 20191213)

RISC-V Instruction Set Manual, Volume II: Privileged Architecture (Privileged Architecture Version 1.13 - ratified 20241017)

RISC-V "C" Compressed Extension (Compressed Architecture Version 1.0)

RISC-V "V" Vector Extension (Vector Architecture Version 1.0 Ratified (riscv-isa-manual tag 20240411 commit 6ee57d8))

RISC-V External Debug Support (RISC-V External Debug Support Version 1.0-STABLE as of commit 83483b1 of 21 August 2023 (this is subject to change))

Chapter 2

Configuration

2.1 Location

This model's VLNV is riscv.ovpworld.org/processor/riscv/1.0.
The model source is usually at:
$IMPERAS_HOME/ImperasLib/source/riscv.ovpworld.org/processor/riscv/1.0
The model binary is usually at:
$IMPERAS_HOME/lib/$IMPERAS_ARCH/ImperasLib/riscv.ovpworld.org/processor/riscv/1.0

2.2 GDB Path

The default GDB for this model is: $IMPERAS_HOME/lib/$IMPERAS_ARCH/gdb/riscv-none-embed-gdb.

2.3 Semi-Host Library

The default semi-host library file is riscv.ovpworld.org/semihosting/pk/1.0

2.4 Processor Endian-ness

This is a LITTLE endian model.

2.5 QuantumLeap Support

This processor is qualified to run in a QuantumLeap enabled simulator.

2.6 Processor ELF code

The ELF code supported by this model is: 0xf3.

Chapter 3

All Variants in this model

This model has these variants

Variant	Description
RV32I	
RV32IM	
RV32IMC	
RV32IMCZce	
RV32IMAC	
RV32G	
RV32GC	
RV32GCZfinx	
RV32GCB	
RV32GCH	
RV32GCK	
RV32GCN	
RV32GCV	
RV32E	
RV32EC	
RV32EM	
RV64I	
RV64IM	
RV64IMC	
RV64IMCZce	
RV64IMAC	
RV64G	
RV64GC	
RV64GCZfinx	
RV64GCB	
RV64GCH	
RV64GCK	
RV64GCN	
RV64GCV	(described in this document)
RVB32I	
RVB32E	

RVB64I	
RVI20U32mandatory	
RVI20U32	
RVI20U64mandatory	
RVI20U64	
RVA20U64mandatory	
RVA20U64	
RVA20S64mandatory	
RVA20S64	
RVA22U64mandatory	
RVA22U64	
RVA22S64mandatory	
RVA22S64	
RVA23U64mandatory	
RVA23U64	
RVA23S64mandatory	
RVA23S64	
RVB23U64mandatory	
RVB23U64	
RVB23S64mandatory	
RVB23S64	

Table 3.1: All Variants in this model

Chapter 4

Bus Master Ports

This model has these bus master ports.

Name	min	max	Connect?	Description
INSTRUCTION	32	64	mandatory	Instruction bus
DATA	32	64	optional	Data bus

Table 4.1: Bus Master Ports

Chapter 5

Bus Slave Ports

This model has no bus slave ports.

Chapter 6

Net Ports

This model has these net ports.

Name	Type	Connect?	Description
reset	input	optional	Reset
reset_addr	input	optional	Externally-applied reset address
nmi	input	optional	NMI
nmi_cause	input	optional	Externally-applied NMI cause
nmi_addr	input	optional	Externally-applied NMI address
SSWInterrupt	input	optional	Supervisor software interrupt
MSWInterrupt	input	optional	Machine software interrupt
STimerInterrupt	input	optional	Supervisor timer interrupt
MTimerInterrupt	input	optional	Machine timer interrupt
SExternalInterrupt	input	optional	Supervisor external interrupt
MExternalInterrupt	input	optional	Machine external interrupt
irq_ack_o	output	optional	Interrupt acknowledge (pulse)
irq_id_o	output	optional	Acknowledged interrupt id (valid during irq_ack_o pulse)
sec_lvl_o	output	optional	Current privilege level
LR_address	output	optional	Port written with effective address for LR instruction
SC_address	output	optional	Port written with effective address for SC instruction
SC_valid	input	optional	SC_address valid input signal
AMO_active	output	optional	Port written with code indicating active AMO
illegalinstr	input	optional	Artifact signal causing Illegal Instruction exception on rising edge
hardwareerror	input	optional	Artifact signal causing Hardware Error exception on rising edge
artifactdata	input	optional	Artifact data value. Sampled on hardwareerror artifact port triggers
deferint	input	optional	Artifact signal causing interrupts to be held off while high

coverpoint	output	optional	Artifact port written with coverage point identifier
readcsr	output	optional	Artifact port written with CSR/GPR information when CSR is read
HaltReason	output	optional	Artifact halt reason indication
core_wfi_mode	output	optional	WFI active indication
restart_wfi	input	optional	Artifact signal causing restart from WFI state when high

Table 6.1: Net Ports

Chapter 7

FIFO Ports

This model has no FIFO ports.

Chapter 8

Formal Parameters

Name	Type	Description
Fundamental		
user_version	Enumeration	Specify required User Architecture version
	2.2	User Architecture Version 2.2
	2.3	Deprecated and equivalent to 20191213
	20190305	Deprecated and equivalent to 20191213
	20191213	User Architecture Version 20191213
priv_version	Enumeration	Specify required Privileged Architecture version
	1.10	Privileged Architecture Version 1.10
	1.11	Privileged Architecture Version 1.11 - ratified 20190608
	1.12	Privileged Architecture Version 1.12 - ratified 20211203
	1.13	Privileged Architecture Version 1.13 - ratified 20241017
	master	Privileged Architecture Master Branch as of commit 2c07aa2 (this is subject to change)
	20190405	Deprecated and equivalent to 1.11
	20190608	Deprecated and equivalent to 1.11
	20211203	Deprecated and equivalent to 1.12
Smepmp_version	Enumeration	Specify required Smepmp Architecture version
	none	Smepmp not implemented
	0.9.5	Smepmp version 0.9.5 (deprecated and identical to 1.0)
	1.0	Smepmp version 1.0
numHarts	Uns32	Specify the number of hart contexts in a multiprocessor
enable_host_atomics	Boolean	Enable use of host atomic instructions to emulate load/store exclusive operations (not architecturally accurate, accelerates parallel simulation)
enable_expanded	Boolean	Specify that 48-bit and 64-bit expanded instructions are supported
endianFixed	Boolean	Specify that data endianness is fixed (mstatus.{MBE,SBE,UBE} fields are read-only)
misa_MXL	Uns32	Override default value of misa.MXL
misa_Extensions	Uns32	Override default value of misa.Extensions
add_Extensions	String	Add extensions specified by letters to misa.Extensions (for example, specify "VD" to add V and D features)

sub_Extensions	String	Remove extensions specified by letters from misa.Extensions (for example, specify "VD" to remove V and D features)
misa_Extensions_mask	Uns32	Override mask of writable bits in misa.Extensions
add_Extensions_mask	String	Add extensions specified by letters to mask of writable bits in misa.Extensions (for example, specify "VD" to add V and D features)
sub_Extensions_mask	String	Remove extensions specified by letters from mask of writable bits in misa.Extensions (for example, specify "VD" to remove V and D features)
add_implicit_Extensions	String	Add extensions specified by letters to implicitly-present extensions not visible in misa.Extensions
sub_implicit_Extensions	String	Remove extensions specified by letters from implicitly-present extensions not visible in misa.Extensions
Vector_Extension		
vector_version	Enumeration	Specify required Vector Architecture version
	0.7.1-draft-20190605	Vector Architecture Version 0.7.1-draft-20190605 (riscv-v-spec commit 9dbf12d)
	0.7.1-draft-20190605+	Vector Architecture Version 0.7.1-draft-20190605 with custom features (not for general use)
	0.8-draft-20190906	Vector Architecture Version 0.8-draft-20190906 (riscv-v-spec commit be942b4)
	0.8-draft-20191004	Vector Architecture Version 0.8-draft-20191004 (riscv-v-spec commit 55bb8f6)
	0.8-draft-20191117	Vector Architecture Version 0.8-draft-20191117 (riscv-v-spec commit 3cb0a35)
	0.8-draft-20191118	Vector Architecture Version 0.8-draft-20191118 (riscv-v-spec commit 812001b)
	0.8	Vector Architecture Version 0.8 (riscv-v-spec commit 9a65519)
	0.9	Vector Architecture Version 0.9 (riscv-v-spec commit cb7d225)
	1.0-draft-20210130	Vector Architecture Version 1.0-draft-20210130 (riscv-v-spec commit 8e768b0
	1.0-rc1-20210608	Vector Architecture Version 1.0-rc1-20210608 (riscv-v-spec commit 795a4dd)
	1.0	Vector Architecture Version 1.0 (riscv-v-spec commit 8af318f)
	1.0-ratified	Vector Architecture Version 1.0 Ratified (riscv-isa-manual tag 20240411 commit 6ee57d8)
	master	Vector Architecture Main Branch github.com/riscv/riscv-isa-manual tag riscv-isa-release-1bec7d3-2024-05-28 (this is subject to change)
unalignedV	Boolean	Specify whether the processor supports unaligned memory accesses for vector instructions
vstart0_non_ld_st	Boolean	Whether CSR vstart must be 0 for non-load/store vector instructions
vstart0_ld_st	Boolean	Whether CSR vstart must be 0 for load/store vector instructions
align_whole	Boolean	Whether whole-register load addresses must be aligned using the encoded EEW
vill_trap	Boolean	Whether illegal vtype values cause trap instead of setting vtype.vill
mstatus_vs_mode	Enumeration	Specify conditions causing update of mstatus.VS to dirty

	state_update	any vector state update sets dirty
	execute_any	state_update plus vector instructions where vl=0 (which do not update state)
mstatus_VS	Uns32	Override default value of mstatus.VS (initial state of vector unit)
ELEN	Uns32	Override ELEN
SLEN	Uns32	Override SLEN (before version 1.0 only)
VLEN	Uns32	Override VLEN
EEW_index	Uns32	Override maximum supported index EEW (use ELEN if zero)
SEW_min	Uns32	Override minimum supported SEW
agnostic_ones	Boolean	Specify that vector agnostic elements are set to 1
agnostic_mask	Boolean	Specify that artifact mask registers holding agnostic bits are present
Zvlsseg	Boolean	Specify that Zvlsseg is implemented
Zvamo	Boolean	Specify that Zvamo is implemented
Zvediv	Boolean	Specify that Zvediv is implemented
Zvqmac	Boolean	Specify that Zvqmac is implemented
Zve32x	Boolean	Specify that Zve32x is implemented
Zve32f	Boolean	Specify that Zve32f is implemented
Zve64x	Boolean	Specify that Zve64x is implemented
Zve64f	Boolean	Specify that Zve64f is implemented
Zve64d	Boolean	Specify that Zve64d is implemented
Zvfh	Boolean	Specify that Zvfh is implemented
Zvfhmin	Boolean	Specify that Zvfhfmin is implemented
Zvfbfmin	Boolean	Specify that Zvfbfmin is implemented
Zvfbfwma	Boolean	Specify that Zvfbfwma is implemented
Compressed_Extension		
compress_version	Enumeration	Specify required Compressed Architecture version
	legacy	Compressed Architecture absent or legacy version
	0.70.1	Compressed Architecture Version 0.70.1
	0.70.5	Compressed Architecture Version 0.70.5
	1.0.0-RC5.7	Compressed Architecture Version 1.0.0-RC5.7
	1.0	Compressed Architecture Version 1.0
Zca	Boolean	Specify that Zca is implemented
Zcb	Boolean	Specify that Zcb is implemented
Zcf	Boolean	Specify that Zcf is implemented
Zcmp	Boolean	Specify that Zcmp is implemented
Zcmt	Boolean	Specify that Zcmt is implemented
Zcmop	Boolean	Specify that Zcmop is implemented
Debug_Extension		
debug_version	Enumeration	Specify required Debug Architecture version
	0.13.2-DRAFT	RISC-V External Debug Support Version 0.13.2-DRAFT
	0.14.0-DRAFT	RISC-V External Debug Support Version 0.14.0-DRAFT
	1.0.0-STABLE	RISC-V External Debug Support Version 1.0.0-STABLE
	1.0-STABLE	RISC-V External Debug Support Version 1.0-STABLE as of commit 83483b1 of 21 August 2023 (this is subject to change)
debug_mode	Enumeration	Specify how Debug mode is implemented (value 'none' implies Sdext is not implemented)
	none	Debug mode not implemented
	vector	Debug mode implemented by execution at vector
	interrupt	Debug mode implemented by interrupt

	halt	Debug mode implemented by halt
	inject	Debug mode implemented using injected instructions
Interrupts_Exceptions		
rnmi_version	Enumeration	Specify required RNMI Architecture version
	none	RNMI not implemented
	0.2.1	RNMI version 0.2.1
	0.4_nmie1	RNMI version 0.4, except mnstatus.nmie=1 at reset
	0.4	RNMI version 0.4
mtvec_is_ro	Boolean	Specify whether mtvec CSR is read-only
tvec_mode_rsvd	Enumeration	Specify behavior of writes of reserved values to xTVEC.mode field (ignored when CLIC_version <20191208)
	TVEC_M_KEEP	Attempt to set mode to 2 keeps previous value
	TVEC_M_CLIC_VECTOR	Attempt to set mode to 2 will be mapped to 3 (Vectored CLIC mode)
tvec_align	Uns32	Specify additional hardware-enforced alignment on writes to mtvec/stvec/utvec that set a non-direct mode (i.e. bit 0 of the write is 1)
ecode_mask	Uns64	Specify hardware-enforced mask of writable bits in xcause.ExceptionCode
ecode_nmi	Uns64	Specify xcause.ExceptionCode for NMI
nmi_absent	Boolean	Specify whether NMI input is absent
nmi_is_latched	Boolean	Specify whether NMI input is latched on rising edge (if False, it is level-sensitive)
nmi_high_priority	Boolean	Specify whether NMI input is higher priority than Debug and Trigger Module events
nmi_update_mstatus	Boolean	Specify whether an NMI exception updates the mstatus CSR mie and mpie fields
nmi_update_tcontrol	Boolean	Specify whether an NMI exception updates the tcontrol CSR mte and mpte fields
nmi_zero_mtval	Boolean	Specify whether an NMI exception writes 0 to the mtval CSR
mtval_is_ro	Boolean	Specify whether mtval CSR is read-only
tval_zero	Boolean	Specify whether mtval/stval/vstval/utval are hard wired to zero
tval_mask	Uns64	Specify write mask for mtval/stval/vstval/utval. Ignored if tval_zero is True, 0 maps to -1ULL
tval_zero_ecodes_mask	Uns64	Specify mask of exception codes that always set mtval to 0. Not used for interrupts, ignored if tval_zero is True
tval_zero_ebreak	Boolean	Specify whether mtval/stval/vstval/utval are set to zero by an ebreak
tval_ii_code	Boolean	Specify whether mtval/stval contain faulting instruction bits on illegal instruction exception
trap_preserves_lr	Boolean	Whether a trap preserves active LR/SC state
xret_preserves_lr	Boolean	Whether an xret instruction preserves active LR/SC state
reset_address	Uns64	Override reset vector address
nmi_address	Uns64	Override NMI vector address
CLINT_address	Uns64	Specify base address of internal CLINT model (or 0 for no CLINT)
local_int_num	Uns32	Specify number of local interrupts (excludes standard set 0 to 15 if basic interrupt mode is present)

unimp_int_mask	Uns64	Specify mask of unimplemented interrupts (e.g. 1<<9 indicates Supervisor external interrupt unimplemented)
force_mideleg	Uns64	Specify mask of interrupts always delegated to lower-priority execution level from Machine execution level
force_sideleg	Uns64	Specify mask of interrupts always delegated to User execution level from Supervisor execution level
no_ideleg	Uns64	Specify mask of interrupts that cannot be delegated to lower-priority execution levels
no_edeleg	Uns64	Specify mask of exceptions that cannot be delegated to lower-priority execution levels
external_int_id	Boolean	Whether to add nets allowing External Interrupt ID codes to be forced
posedge_sswi	Boolean	Specify that Supervisor SW Interrupt input is edge-triggered (required when driven from ACLINT)
Fast_Interrupt		
CLIC_version	Enumeration	Specify required CLIC version
	20180831	CLIC Version 20180831 (commit 6d369ab)
	0.9-draft-20191208	CLIC Version 0.9-draft-20191208 (commit f9ab734)
	0.9-draft-20220315	CLIC Version 0.9-draft-20220315 (commit 45a2e91)
	0.9-draft-20221108	CLIC Version 0.9-draft-20221108 (commit fdcd068)
	0.9-draft-20230801	CLIC Version 0.9-draft-20230801 (commit 97a2d57)
	0.10-draft-20250317	CLIC Version 0.10-draft-20250317 (commit 62092fc)
	master	CLIC Master Branch as of commit 62092fc (this is subject to change)
CLICLEVELS	Uns32	Specify number of interrupt levels implemented by CLIC, or 0 if CLIC absent
WorldGuard		
WG_version	Enumeration	Specify required WorldGuard version
	none	WorldGuard not implemented
	0.4	WorldGuard Version 0.4
Floating_Point		
fp16_version	Enumeration	Specify required 16-bit floating point format
	none	No 16-bit floating point implemented
	IEEE754	IEEE 754 half precision implemented
	BFLOAT16	BFLOAT16 implemented
	dynamic	Dynamic 16-bit floating point implemented
mstatus_fs_mode	Enumeration	Specify conditions causing update of mstatus.FS to dirty
	write_1	Any non-zero flag result sets mstatus.fs dirty
	write_any	Any write of flags sets mstatus.fs dirty
	execute_not_store	Any floating point non-store instruction sets mstatus.fs dirty
	execute_any	Any floating point instruction sets mstatus.fs dirty
	always_dirty	mstatus.fs and vs are either off or dirty
	force_dirty	mstatus.fs is forced to dirty
d_requires_f	Boolean	If D and F extensions are separately enabled in the misa CSR, whether D is enabled only if F is enabled
enable_fflags_i	Boolean	Whether fflags_i artifact register present (shows per-instruction floating point flags)
enable_DAZ	Boolean	Whether subnormal floating point operands are flushed to zero (non-compliant option)
enable_FZ	Boolean	Whether subnormal floating point results are flushed to zero (non-compliant option)

mstatus_FS	Uns32	Override default value of mstatus.FS (initial state of floating point unit)
Zfa	Boolean	Specify that Zfa is implemented (additional floating point instructions)
Zfh	Boolean	Specify that Zfh is implemented (IEEE half-precision floating point is supported)
Zfhmin	Boolean	Specify that Zfhmin is implemented (restricted IEEE half-precision floating point is supported)
Zfbfmin	Boolean	Specify that Zfbfmin is implemented (restricted BFLOAT16 floating point is supported)
Zfinx_version	Enumeration	Specify version of Zfinx implemented (use integer register file for floating point instructions)
	none	Zfinx not implemented
	0.4	Zfinx version 0.4
	0.41	Zfinx version 0.41
	1.0	Zfinx version 1.0
Memory		
lr_sc_constraint	Enumeration	Specify memory constraint for LR/SC instructions
	none	Memory access not constrained
	user1	Memory access constrained by MEM_CONSTRAINT_USER1
	user2	Memory access constrained by MEM_CONSTRAINT_USER2
	user3	Memory access constrained by MEM_CONSTRAINT_USER3
	user4	Memory access constrained by MEM_CONSTRAINT_USER4
amo_constraint	Enumeration	Specify memory constraint for AMO instructions
	none	Memory access not constrained
	user1	Memory access constrained by MEM_CONSTRAINT_USER1
	user2	Memory access constrained by MEM_CONSTRAINT_USER2
	user3	Memory access constrained by MEM_CONSTRAINT_USER3
	user4	Memory access constrained by MEM_CONSTRAINT_USER4
push_pop_constraint	Enumeration	Specify memory constraint for PUSH/POP instructions
	none	Memory access not constrained
	user1	Memory access constrained by MEM_CONSTRAINT_USER1
	user2	Memory access constrained by MEM_CONSTRAINT_USER2
	user3	Memory access constrained by MEM_CONSTRAINT_USER3
	user4	Memory access constrained by MEM_CONSTRAINT_USER4
vector_constraint	Enumeration	Specify memory constraint for vector load/store instructions
	none	Memory access not constrained
	user1	Memory access constrained by MEM_CONSTRAINT_USER1
	user2	Memory access constrained by MEM_CONSTRAINT_USER2

	user3	Memory access constrained by MEM_CONSTRAINT_USER3
	user4	Memory access constrained by MEM_CONSTRAINT_USER4
updatePTEA	Boolean	Specify whether hardware update of PTE A bit is supported (ignored when parameter Svadu is True)
updatePTED	Boolean	Specify whether hardware update of PTE D bit is supported (ignored when parameter Svadu is True)
unaligned_low_pri	Boolean	Specify whether address misaligned exceptions are lower priority than page or access fault exceptions
unaligned	Boolean	Specify whether the processor supports unaligned memory accesses
Zam	Boolean	Specify whether the processor supports unaligned memory accesses for AMO instructions
unalignedPushPop	Boolean	Specify whether the processor supports unaligned memory accesses for push/pop (deprecated, ignored when unaligned=True)
aligned_uncached_PBMT	Boolean	Specify whether pages with non-zero PBMT require aligned access (Svpbmt extension only)
amo_aborts_lr_sc	Boolean	Specify whether AMO operations abort any active LR/SC pair
ASID_bits	Uns32	Specify the number of implemented ASID bits
lr_sc_grain	Uns32	Specify byte granularity of LR/SC lock region (constrained to a power of two)
lr_sc_match_size	Boolean	Whether LR/SC access sizes must match
lr_sc_use_pa	Boolean	Whether LR/SC uses physical addresses, so reservations are supported across virtual aliases
lr_sc_abort_self	Boolean	Whether LR/SC can be aborted by a store on the same hart that executed the LR instruction
ignore_non_leaf_DAU	Boolean	Whether non-zero D, A and U bits in non-leaf PTEs are ignored (if False, a trap is taken)
MPU_grain	Uns32	Specify Ssmpu region granularity, G (0 =>4 bytes, 1 =>8 bytes, etc)
MPU_registers	Uns32	Specify the number of implemented Ssmpu address registers
MPU_decompose	Boolean	Whether unaligned Ssmpu accesses are decomposed into separate aligned accesses
Sv_modes	Uns32	Specify bit mask of implemented address translation modes (e.g. (1<<0)+(1<<8) indicates “bare” and “Sv39” modes may be selected in satp.MODE)
Simulation_Artifact		
leaf_hart_prefix	String	Specify string name prefix for harts in a cluster
mtime_counter	String	Specify name of shared mtime counter
mtime_Hz	Double	Specify mtime counter frequency
mtime_bits	Uns32	Specify mtime counter bit width
use_hw_reg_names	Boolean	Specify whether to use hardware register names x0-x31 and f0-f31 instead of ABI register names
no_pseudo_inst	Boolean	Specify whether pseudo-instructions should not be reported in trace and disassembly
show_c_prefix	Boolean	Specify whether compressed instruction prefix should be reported in trace and disassembly
ABI_d	Boolean	Specify whether D registers are used for parameters (ABI SemiHosting)
verbose	Boolean	Specify verbose output messages
traceVolatile	Boolean	Specify whether volatile registers (e.g. minstret) should be shown in change trace

wfi_restart	Boolean	Specify whether to automatically restart from WFI state when model is simulated
enable_CSR_bus	Boolean	Add artifact CSR bus port, allowing CSR registers to be externally implemented
CSR_remap	String	Comma-separated list of CSR number mappings, each of the form <csrName>=<number>
fmv_old_s	Boolean	Whether FMV instruction disassembly uses obsolete 'S' naming instead of 'W' [Deprecated - will be removed in the future]
ASID_cache_size	Uns32	Specify the number of different ASIDs for which TLB entries are cached; a value of 0 implies no limit
Instruction_CSR_Behavior		
wfi_is_nop	Boolean	Specify whether WFI should be treated as a NOP (if not, halt while waiting for interrupts)
wfi_resume_not_trap	Boolean	Specify whether pending wakeup events should cause WFI to be treated as a NOP instead of taking a trap
TW_time_limit	Uns32	Specify nominal cycle timeout for instructions controlled by mstatus.TW
PAUSE_time_limit	Uns32	Specify nominal cycle timeout for PAUSE instruction
counteren_mask	Uns32	Specify hardware-enforced mask of writable bits in mcounteren/scounteren registers
scounteren_zero_mask	Uns32	Specify hardware-enforced mask of always-zero bits in scounteren register
noinhibit_mask	Uns32	Specify hardware-enforced mask of always-zero bits in mcountinhibit register
cycle_undefined	Boolean	Specify that the cycle CSR is undefined
mcycle_undefined	Boolean	Specify that the mcycle CSR is undefined
time_undefined	Boolean	Specify that the time CSR is undefined
instret_undefined	Boolean	Specify that the instret CSR is undefined
minstret_undefined	Boolean	Specify that the minstret CSR is undefined
hpmcounter_undefined	Boolean	Specify that the hpmcounter CSRs are undefined
mhpmcounter_undefined	Boolean	Specify that the mhpmcounter CSRs are undefined
CSR_Masks		
mtvec_mask	Uns64	Specify hardware-enforced mask of writable bits in mtvec register
stvec_mask	Uns64	Specify hardware-enforced mask of writable bits in stvec register
jvt_mask	Uns64	Specify hardware-enforced mask of writable bits in Zcmt jvt register
tdata1_mask	Uns64	Specify hardware-enforced mask of writable bits in Trigger Module tdata1 register
mip_mask	Uns64	Specify hardware-enforced mask of writable bits in mip register
sip_mask	Uns64	Specify hardware-enforced mask of writable bits in sip register
envcfg_mask	Uns64	Specify hardware-enforced mask of writable bits in envcfg registers
mtvec_sext	Boolean	Specify whether mtvec is sign-extended from most-significant bit
stvec_sext	Boolean	Specify whether stvec is sign-extended from most-significant bit
SXL_writable	Boolean	Specify that mstatus.SXL is writable (feature under development)

UXL_writable	Boolean	Specify that mstatus.UXL is writable (feature under development)
csrindCustom	Boolean	Force MSB of *iselect CSRs to be writable to support custom extensions using Smcsrind
Trigger		
tdata2_undefined	Boolean	Specify that the tdata2 CSR is undefined
tdata3_undefined	Boolean	Specify that the tdata3 CSR is undefined
tinfo_undefined	Boolean	Specify that the tinfo CSR is undefined
tcontrol_undefined	Boolean	Specify that the tcontrol CSR is undefined
mcontext_undefined	Boolean	Specify that the mcontext CSR is undefined
scontext_undefined	Boolean	Specify that the scontext CSR is undefined
mscontext_undefined	Boolean	Specify that the mscontext CSR is undefined (Debug Version 0.14.0 and later)
amo_trigger	Boolean	Specify whether AMO load/store operations activate triggers
no_hit	Boolean	Specify that tdata1.hit* bits are unimplemented
no_sselect_2	Boolean	Specify that textra.sselect=2 is not supported (no trigger match by ASID)
trigger_num	Uns32	Specify the number of implemented hardware triggers (0 implies Sdtrig is not implemented)
mask_tselect	Boolean	Whether values written to tselect are masked according to the trigger_num setting
tinfo	Uns32	Override tinfo register (for all triggers)
tinfo0	Uns32	Specify value of read-only tinfo for trigger 0
tinfo1	Uns32	Specify value of read-only tinfo for trigger 1
tinfo2	Uns32	Specify value of read-only tinfo for trigger 2
tinfo3	Uns32	Specify value of read-only tinfo for trigger 3
tinfo4	Uns32	Specify value of read-only tinfo for trigger 4
tinfo5	Uns32	Specify value of read-only tinfo for trigger 5
tinfo6	Uns32	Specify value of read-only tinfo for trigger 6
tinfo7	Uns32	Specify value of read-only tinfo for trigger 7
tinfo8	Uns32	Specify value of read-only tinfo for trigger 8
tinfo9	Uns32	Specify value of read-only tinfo for trigger 9
tinfo10	Uns32	Specify value of read-only tinfo for trigger 10
tinfo11	Uns32	Specify value of read-only tinfo for trigger 11
tinfo12	Uns32	Specify value of read-only tinfo for trigger 12
tinfo13	Uns32	Specify value of read-only tinfo for trigger 13
tinfo14	Uns32	Specify value of read-only tinfo for trigger 14
tinfo15	Uns32	Specify value of read-only tinfo for trigger 15
trigger_match	Uns32	Specify legal “match” values for triggers of type 2 and 6 (bitmask)
mcontext_bits	Uns32	Specify the number of implemented bits in mcontext
scontext_bits	Uns32	Specify the number of implemented bits in scontext
mvalue_bits	Uns32	Specify the number of implemented bits in textra.mvalue (if zero, textra.mselect is tied to zero)
svalue_bits	Uns32	Specify the number of implemented bits in textra.svalue (if zero, textra.sselect is tied to zero)
mcontrol_maskmax	Uns32	Specify mcontrol.maskmax value
chain_tval	Enumeration	Specify which trigger provides xtval when triggers are chained
	first	first matching trigger provides xtval
	last	last matching trigger provides xtval
	first_non_epc	first matching trigger provides xtval (prefer non-epc)
	last_non_epc	last matching trigger provides xtval (prefer non-epc)

mcontrol_addl_comp	Boolean	whether mcontrol compare for select=0 includes recommended additional values
PMP Configuration		
PMP_grain	Uns32	Specify PMP region granularity, G (0 =>4 bytes, 1 =>8 bytes, etc)
PMP_registers	Uns32	Specify the number of supported PMP regions
PMP_csrs	Uns32	Specify the number of PMP address CSRs (are RAZ/WI if corresponding region is not implemented)
PMP_max_page	Uns32	Specify the maximum size of PMP region to map if non-zero (may improve performance; constrained to a power of two)
PMP_decompose	Boolean	Whether unaligned PMP accesses are decomposed into separate aligned accesses
PMP_undefined	Boolean	Whether accesses to unimplemented PMP registers are undefined (if True) or write ignored and zero (if False)
PMP_R0W1	Enumeration	Specify behavior of PMPiCFG RWX field on illegal write with R=0 and W=1
	RWX_00X	set R=0 and W=0, modify X
	RWX_00P	set R=0 and W=0, preserve X
	RWX_11X	set R=1 and W=1, modify X
	RWX_PPX	preserve previous R and W, modify X
	RWX_PPP	preserve previous RWX
	RWX_000	set RWX=000
PMP_A_RSVD	Enumeration	Specify behavior of writes of reserved values to PMPiCFG.A
	PMP_A_NONE	No reserved values for PMPiCFG.A
	PMP_NA4_OFF	Attempts to write NA4 mode to PMPiCFG.A will be mapped to OFF
PMP_maskparams	Boolean	Enable parameters to change the read-only masks for PMP CSRs
PMP_initialparams	Boolean	Enable parameters to change the reset values for PMP CSRs
Sspmp	Boolean	Specify that Sspmp is implemented
mpmpdeleg_min	Uns32	Specify minimum value of mpmpdeleg (if Sspmp enabled)
mpmpdeleg_max	Uns32	Specify maximum value of mpmpdeleg (if Sspmp enabled)
Other_Extensions		
Svnapot_page_mask	Uns64	Specify mask of implemented Svnapot intermediate page sizes (e.g. 1<<16 means 64KiB contiguous regions are supported)
Smstateen	Boolean	Specify that Smstateen is implemented (ignored when priv_version <1.12)
Ssstateen	Boolean	Specify that Ssstateen is implemented (implied when Smstateen is True, ignored when priv_version <1.12)
Smcsrind	Boolean	Specify that Smcsrind is implemented (plus Sscsrind if S extension is implemented)
Sstc	Boolean	Specify that Sstc is implemented
Sscofpmf	Boolean	Specify that Sscofpmf is implemented
Ssdbltrp	Boolean	Specify that Ssdbltrp is implemented
Smdbltrp	Boolean	Specify that Smdbltrp is implemented
Smcntrpmf	Boolean	Specify that Smcntrpmf is implemented
Smcdeleg	Boolean	Specify that Smcdeleg is implemented

Svpbmt	Boolean	Specify that Svpbmt is implemented
Ssnpm	Enumeration	Specify Ssnpm implemented modes
	none	pointer masking not implemented
	48	PM=XLEN-48 is implemented
	57	PM=XLEN-57 is implemented
	48_57	PM=XLEN-48 and PM=XLEN-57 are implemented
Smnpm	Enumeration	Specify Smnpm implemented modes
	none	pointer masking not implemented
	48	PM=XLEN-48 is implemented
	57	PM=XLEN-57 is implemented
	48_57	PM=XLEN-48 and PM=XLEN-57 are implemented
Smmpm	Enumeration	Specify Smmpm implemented modes
	none	pointer masking not implemented
	48	PM=XLEN-48 is implemented
	57	PM=XLEN-57 is implemented
	48_57	PM=XLEN-48 and PM=XLEN-57 are implemented
Svinval	Boolean	Specify that Svinval is implemented
Svadu	Boolean	Specify that Svadu is implemented
Ssqosid	Boolean	Specify that Ssqosid is implemented
Zihintntl	Boolean	Specify that Zihintntl is implemented (instruction decode only, implemented as NOP)
Zihintpause	Boolean	Specify that Zihintpause is implemented
Zicond	Boolean	Specify that Zicond is implemented
Zicsr	Boolean	Specify that Zicsr is implemented
Zifencei	Boolean	Specify that Zifencei is implemented
Zicbom	Boolean	Specify that Zicbom is implemented
Zicbop	Boolean	Specify that Zicbop is implemented
Zicboz	Boolean	Specify that Zicboz is implemented
Zimop	Boolean	Specify that Zimop is implemented
Zicfiss	Boolean	Specify that Zicfiss is implemented
Zicfilp	Boolean	Specify that Zicfilp is implemented
Zibi	Boolean	Specify that Zibi is implemented
Zaamo	Boolean	Specify that Zaamo is implemented
Zalrsc	Boolean	Specify that Zalrsc is implemented
Zacas	Boolean	Specify that Zacas is implemented
Zabha	Boolean	Specify that Zabha is implemented
Zawrs	Boolean	Specify that Zawrs is implemented
Zmmul	Boolean	Specify that Zmmul is implemented
Bit_Manipulation_Extension		
misa_B_Zba_Zbb_Zbs	Boolean	Specify that misa.B is present if Zba, Zbb and Zbs are all implemented (from bit manipulation version 1.0.0 only; ignored for previous versions)
Zba	Boolean	Specify that Zba is implemented
Zbb	Boolean	Specify that Zbb is implemented
Zbc	Boolean	Specify that Zbc is implemented
Zbe	Boolean	Specify that Zbe is implemented (ignored if version 1.0.0)
Zbf	Boolean	Specify that Zbf is implemented (ignored if version 1.0.0)
Zbm	Boolean	Specify that Zbm is implemented (ignored if version 1.0.0)
Zbp	Boolean	Specify that Zbp is implemented (ignored if version 1.0.0)
Zbr	Boolean	Specify that Zbr is implemented (ignored if version 1.0.0)
Zbs	Boolean	Specify that Zbs is implemented

Zbt	Boolean	Specify that Zbt is implemented (ignored if version 1.0.0)
CSR_Defaults		
mvendorid	Uns64	Override mvendorid register
marchid	Uns64	Override marchid register
mimpid	Uns64	Override mimpid register
mhartid	Uns64	Override mhartid register (or first mhartid of an incrementing sequence if this is an SMP variant)
mconfigptr	Uns64	Override mconfigptr register
mtvec	Uns64	Override mtvec register
mseccfg	Uns64	Override mseccfg register
menvcfg	Uns64	Override menvcfg register
AIA_Interrupts		
Smaia	Boolean	Specify that Smaia CSRs are present

Table 8.1: Parameters that can be set in: Hart

8.1 Parameters with enumerated types

8.1.1 Parameter user_version

Set to this value	Description
2.2	User Architecture Version 2.2
2.3	Deprecated and equivalent to 20191213
20190305	Deprecated and equivalent to 20191213
20191213	User Architecture Version 20191213

Table 8.2: Values for Parameter user_version

8.1.2 Parameter priv_version

Set to this value	Description
1.10	Privileged Architecture Version 1.10
1.11	Privileged Architecture Version 1.11 - ratified 20190608
1.12	Privileged Architecture Version 1.12 - ratified 20211203
1.13	Privileged Architecture Version 1.13 - ratified 20241017
master	Privileged Architecture Master Branch as of commit 2c07aa2 (this is subject to change)
20190405	Deprecated and equivalent to 1.11
20190608	Deprecated and equivalent to 1.11
20211203	Deprecated and equivalent to 1.12

Table 8.3: Values for Parameter priv_version

8.1.3 Parameter vector_version

Set to this value	Description
0.7.1-draft-20190605	Vector Architecture Version 0.7.1-draft-20190605 (riscv-v-spec commit 9dbf12d)
0.7.1-draft-20190605+	Vector Architecture Version 0.7.1-draft-20190605 with custom features (not for general use)
0.8-draft-20190906	Vector Architecture Version 0.8-draft-20190906 (riscv-v-spec commit be942b4)
0.8-draft-20191004	Vector Architecture Version 0.8-draft-20191004 (riscv-v-spec commit 55bb8f6)

0.8-draft-20191117	Vector Architecture Version 0.8-draft-20191117 (riscv-v-spec commit 3cb0a35)
0.8-draft-20191118	Vector Architecture Version 0.8-draft-20191118 (riscv-v-spec commit 812001b)
0.8	Vector Architecture Version 0.8 (riscv-v-spec commit 9a65519)
0.9	Vector Architecture Version 0.9 (riscv-v-spec commit cb7d225)
1.0-draft-20210130	Vector Architecture Version 1.0-draft-20210130 (riscv-v-spec commit 8e768b0
1.0-rc1-20210608	Vector Architecture Version 1.0-rc1-20210608 (riscv-v-spec commit 795a4dd)
1.0	Vector Architecture Version 1.0 (riscv-v-spec commit 8af318f)
1.0-ratified	Vector Architecture Version 1.0 Ratified (riscv-isa-manual tag 20240411 commit 6ee57d8)
master	Vector Architecture Main Branch github.com/riscv/riscv-isa-manual tag riscv-isa-release-1bec7d3-2024-05-28 (this is subject to change)

Table 8.4: Values for Parameter vector_version

8.1.4 Parameter compress_version

Set to this value	Description
legacy	Compressed Architecture absent or legacy version
0.70.1	Compressed Architecture Version 0.70.1
0.70.5	Compressed Architecture Version 0.70.5
1.0.0-RC5.7	Compressed Architecture Version 1.0.0-RC5.7
1.0	Compressed Architecture Version 1.0

Table 8.5: Values for Parameter compress_version

8.1.5 Parameter debug_version

Set to this value	Description
0.13.2-DRAFT	RISC-V External Debug Support Version 0.13.2-DRAFT
0.14.0-DRAFT	RISC-V External Debug Support Version 0.14.0-DRAFT
1.0.0-STABLE	RISC-V External Debug Support Version 1.0.0-STABLE
1.0-STABLE	RISC-V External Debug Support Version 1.0-STABLE as of commit 83483b1 of 21 August 2023 (this is subject to change)

Table 8.6: Values for Parameter debug_version

8.1.6 Parameter rnmi_version

Set to this value	Description
none	RNMI not implemented
0.2.1	RNMI version 0.2.1
0.4_nmie1	RNMI version 0.4, except mnstatus.nmie=1 at reset
0.4	RNMI version 0.4

Table 8.7: Values for Parameter rnmi_version

8.1.7 Parameter Smepmp_version

Set to this value	Description
none	Smepmp not implemented
0.9.5	Smepmp version 0.9.5 (deprecated and identical to 1.0)
1.0	Smepmp version 1.0

Table 8.8: Values for Parameter Smepmp_version

8.1.8 Parameter CLIC_version

Set to this value	Description
20180831	CLIC Version 20180831 (commit 6d369ab)
0.9-draft-20191208	CLIC Version 0.9-draft-20191208 (commit f9ab734)
0.9-draft-20220315	CLIC Version 0.9-draft-20220315 (commit 45a2e91)
0.9-draft-20221108	CLIC Version 0.9-draft-20221108 (commit fdcd068)
0.9-draft-20230801	CLIC Version 0.9-draft-20230801 (commit 97a2d57)
0.10-draft-20250317	CLIC Version 0.10-draft-20250317 (commit 62092fc)
master	CLIC Master Branch as of commit 62092fc (this is subject to change)

Table 8.9: Values for Parameter CLIC_version

8.1.9 Parameter WG_version

Set to this value	Description
none	WorldGuard not implemented
0.4	WorldGuard Version 0.4

Table 8.10: Values for Parameter WG_version

8.1.10 Parameter fp16_version

Set to this value	Description
none	No 16-bit floating point implemented
IEEE754	IEEE 754 half precision implemented
BFLOAT16	BFLOAT16 implemented
dynamic	Dynamic 16-bit floating point implemented

Table 8.11: Values for Parameter fp16_version

8.1.11 Parameter mstatus_fs_mode

Set to this value	Description
write_1	Any non-zero flag result sets mstatus.fs dirty
write_any	Any write of flags sets mstatus.fs dirty
execute_not_store	Any floating point non-store instruction sets mstatus.fs dirty
execute_any	Any floating point instruction sets mstatus.fs dirty
always_dirty	mstatus.fs and vs are either off or dirty
force_dirty	mstatus.fs is forced to dirty

Table 8.12: Values for Parameter mstatus_fs_mode

8.1.12 Parameter debug_mode

Set to this value	Description
none	Debug mode not implemented

vector	Debug mode implemented by execution at vector
interrupt	Debug mode implemented by interrupt
halt	Debug mode implemented by halt
inject	Debug mode implemented using injected instructions

Table 8.13: Values for Parameter debug_mode

8.1.13 Parameter lr_sc_constraint

Set to this value	Description
none	Memory access not constrained
user1	Memory access constrained by MEM_CONSTRAINT_USER1
user2	Memory access constrained by MEM_CONSTRAINT_USER2
user3	Memory access constrained by MEM_CONSTRAINT_USER3
user4	Memory access constrained by MEM_CONSTRAINT_USER4

Table 8.14: Values for Parameter lr_sc_constraint

8.1.14 Parameter amo_constraint

Set to this value	Description
none	Memory access not constrained
user1	Memory access constrained by MEM_CONSTRAINT_USER1
user2	Memory access constrained by MEM_CONSTRAINT_USER2
user3	Memory access constrained by MEM_CONSTRAINT_USER3
user4	Memory access constrained by MEM_CONSTRAINT_USER4

Table 8.15: Values for Parameter amo_constraint

8.1.15 Parameter push_pop_constraint

Set to this value	Description
none	Memory access not constrained
user1	Memory access constrained by MEM_CONSTRAINT_USER1
user2	Memory access constrained by MEM_CONSTRAINT_USER2
user3	Memory access constrained by MEM_CONSTRAINT_USER3
user4	Memory access constrained by MEM_CONSTRAINT_USER4

Table 8.16: Values for Parameter push_pop_constraint

8.1.16 Parameter vector_constraint

Set to this value	Description
none	Memory access not constrained
user1	Memory access constrained by MEM_CONSTRAINT_USER1
user2	Memory access constrained by MEM_CONSTRAINT_USER2
user3	Memory access constrained by MEM_CONSTRAINT_USER3
user4	Memory access constrained by MEM_CONSTRAINT_USER4

Table 8.17: Values for Parameter vector_constraint

8.1.17 Parameter tvec_mode_rsvd

Set to this value	Description
TVEC_M_KEEP	Attempt to set mode to 2 keeps previous value
TVEC_M_CLIC_VECTOR	Attempt to set mode to 2 will be mapped to 3 (Vectored CLIC mode)

Table 8.18: Values for Parameter tvec_mode_rsvd

8.1.18 Parameter mstatus_vs_mode

Set to this value	Description
state_update	any vector state update sets dirty
execute_any	state_update plus vector instructions where vl=0 (which do not update state)

Table 8.19: Values for Parameter mstatus_vs_mode

8.1.19 Parameter chain_tval

Set to this value	Description
first	first matching trigger provides xtval
last	last matching trigger provides xtval
first_non_epc	first matching trigger provides xtval (prefer non-epc)
last_non_epc	last matching trigger provides xtval (prefer non-epc)

Table 8.20: Values for Parameter chain_tval

8.1.20 Parameter PMP_R0W1

Set to this value	Description
RWX_00X	set R=0 and W=0, modify X
RWX_00P	set R=0 and W=0, preserve X
RWX_11X	set R=1 and W=1, modify X
RWX_PPX	preserve previous R and W, modify X
RWX_PPP	preserve previous RWX
RWX_000	set RWX=000

Table 8.21: Values for Parameter PMP_R0W1

8.1.21 Parameter PMP_A_RSVD

Set to this value	Description
PMP_A_NONE	No reserved values for PMPiCFG.A
PMP_NA4_OFF	Attempts to write NA4 mode to PMPiCFG.A will be mapped to OFF

Table 8.22: Values for Parameter PMP_A_RSVD

8.1.22 Parameter Ssnpm

Set to this value	Description
none	pointer masking not implemented

48	PM=XLEN-48 is implemented
57	PM=XLEN-57 is implemented
48_57	PM=XLEN-48 and PM=XLEN-57 are implemented

Table 8.23: Values for Parameter Ssnpm

8.1.23 Parameter Smnpm

Set to this value	Description
none	pointer masking not implemented
48	PM=XLEN-48 is implemented
57	PM=XLEN-57 is implemented
48_57	PM=XLEN-48 and PM=XLEN-57 are implemented

Table 8.24: Values for Parameter Smnpm

8.1.24 Parameter Smmpm

Set to this value	Description
none	pointer masking not implemented
48	PM=XLEN-48 is implemented
57	PM=XLEN-57 is implemented
48_57	PM=XLEN-48 and PM=XLEN-57 are implemented

Table 8.25: Values for Parameter Smmpm

8.1.25 Parameter Zfinx_version

Set to this value	Description
none	Zfinx not implemented
0.4	Zfinx version 0.4
0.41	Zfinx version 0.41
1.0	Zfinx version 1.0

Table 8.26: Values for Parameter Zfinx_version

8.2 Parameter values and limits

These are the formal parameter limits and actual parameter values

Name	Min	Max	Default	Actual
Fundamental				
variant				RV64GCV
user_version			20191213	20191213
priv_version			1.13	1.13
Smepmp_version			none	none
numHarts	0	0x400	0	0
endian			none	none
enable_host_atomics			f	f

enable_expanded			f	f
endianFixed			f	f
misa_MXL	1	2	2	2
misa_Extensions	0	0x3ffffff	0x34112d	0x34112d
add_Extensions				
sub_Extensions				
misa_Extensions_mask	0	0x3ffffff	0x20112d	0x20112d
add_Extensions_mask				
sub_Extensions_mask				
add_implicit_Extensions				
sub_implicit_Extensions				
Vector_Extension				
vector_version			1.0-ratified	1.0-ratified
unalignedV			f	f
vstart0_non_ld_st			f	f
vstart0_ld_st			f	f
align_whole			f	f
vill_trap			f	f
mstatus_vs_mode			state_update	state_update
mstatus_VS	0	3	0	0
ELEN	32	64	64	64
SLEN	32	0x10000	64	64
VLEN	32	0x10000	0x200	0x200
EEW_index	0	64	0	0
SEW_min	8	64	8	8
agnostic_ones			f	f
agnostic_mask			f	f
Zvlsseg			t	t
Zvamo			t	t
Zvediv			f	f
Zvqmac			t	t
Zve32x			f	f
Zve32f			f	f
Zve64x			f	f
Zve64f			f	f
Zve64d			f	f
Zvfh			f	f
Zvfhmin			f	f
Zvfbfmin			f	f
Zvfbfwma			f	f
Compressed_Extension				
compress_version			1.0	1.0
Zca			t	t
Zcb			f	f
Zcf			t	t

Zcmp			f	f
Zcmt			f	f
Zcmop			f	f
Debug_Extension				
debug_version			1.0-STABLE	1.0-STABLE
debug_mode			none	none
Interrupts_Exceptions				
rnmi_version			none	none
mtvec_is_ro			f	f
tvec_mode_rsvd			TVEC_M_KEEP	TVEC_M_KEEP
tvec_align	0	0x10000	0	0
ecode_mask	0	0xffffffffffffff	0x7fffffffffffff	0x7fffffffffffff
ecode_nmi	0	0xffffffffffffff	0	0
nmi_absent			f	f
nmi_is_latched			f	f
nmi_high_priority			f	f
nmi_update_mstatus			f	f
nmi_update_tcontrol			f	f
nmi_zero_mtval			f	f
mtval_is_ro			f	f
tval_zero			f	f
tval_mask	0	0xffffffffffffff	0	0
tval_zero_ecodes_mask	0	0xffffffffffffff	0	0
tval_zero_ebreak			f	f
tval_ii_code			t	t
trap_preserves_lr			f	f
xret_preserves_lr			f	f
reset_address	0	0xffffffffffffff	0	0
nmi_address	0	0xffffffffffffff	0	0
CLINT_address	0	0xffffffffffffff	0	0
local_int_num	0	48	0	0
unimp_int_mask	0	0xffffffffffffff	0	0
force_mideleg	0	0xffffffffffffff	0	0
force_sideleg	0	0xffffffffffffff	0	0
no_ideleg	0	0xffffffffffffff	0	0
no_edeleg	0	0xffffffffffffff	0	0
external_int_id			f	f
posedge_sswi			f	f
Fast_Interrupt				
CLIC_version			0.10-draft-20250317	0.10-draft-20250317
CLICLEVELS	0	0x100	0	0
WorldGuard				
WG_version			none	none
Floating_Point				
fp16_version			none	none

mstatus_fs_mode			write_1	write_1
d_requires_f			f	f
enable_fflags_i			f	f
enable_DAZ			f	f
enable_FZ			f	f
mstatus_FS	0	3	0	0
Zfa			f	f
Zfh			f	f
Zfhmin			f	f
Zfbfmin			f	f
Zfinx_version			none	none
Memory				
lr_sc_constraint			user1	user1
amo_constraint			user1	user1
push_pop_constraint			user2	user2
vector_constraint			none	none
updatePTEA			f	f
updatePTED			f	f
unaligned_low_pri			f	f
unaligned			f	f
Zam			f	f
unalignedPushPop			f	f
aligned_uncached_PBMT			f	f
amo_aborts_lr_sc			f	f
ASID_bits	0	16	16	16
lr_sc_grain	1	0x10000	1	1
lr_sc_match_size			f	f
lr_sc_use_pa			f	f
lr_sc_abort_self			f	f
ignore_non_leaf_DAU			f	f
MPU_grain	0	29	0	0
MPU_registers	0	64	0	0
MPU_decompose			f	f
Sv_modes	0	0xffff	0x701	0x701
Simulation_Artifact				
leaf_hart_prefix			hart	hart
mtime_counter			mtime_counter	mtime_counter
mtime_Hz	0.000000e+00	1.000000e+09	1.000000e+06	1.000000e+06
mtime_bits	0	64	64	64
use_hw_reg_names			f	f
no_pseudo_inst			f	f
show_c_prefix			f	f
ABI_d			t	t
verbose			f	f
traceVolatile			f	f

wfi_restart			f	f
enable_CSR_bus			f	f
CSR_remap				
fmv_old_s			f	f
ASID_cache_size	0	0x100	8	8
Instruction_CSR_Behavior				
wfi_is_nop			f	f
wfi_resume_not_trap			f	f
TW_time_limit	0	0xffffffff	0	0
PAUSE_time_limit	0	0xffffffff	0	0
counteren_mask	0	0xffffffff	0xffffffff	0xffffffff
scounteren_zero_mask	0	0xffffffff	0	0
noinhibit_mask	0	0xffffffff	0	0
cycle_undefined			f	f
mcycle_undefined			f	f
time_undefined			f	f
instret_undefined			f	f
minstret_undefined			f	f
hpmcounter_undefined			f	f
mhpmcounter_undefined			f	f
CSR_Masks				
mtvec_mask	0	0xfffffffffffff	0	0
stvec_mask	0	0xfffffffffffff	0	0
jvt_mask	0	0xfffffffffffff	0xfffffffffffffc0	0xfffffffffffffc0
tdata1_mask	0	0xfffffffffffff	0xfffffffffffff	0xfffffffffffff
mip_mask	0	0xfffffffffffff	0x337	0x337
sip_mask	0	0xfffffffffffff	0x103	0x103
envcfg_mask	0	0xfffffffffffff	0	0
mtvec_sext			f	f
stvec_sext			f	f
SXL_writable			f	f
UXL_writable			f	f
csrindCustom			f	f
Trigger				
tdata2_undefined			f	f
tdata3_undefined			f	f
tinfo_undefined			f	f
tcontrol_undefined			f	f
mcontext_undefined			f	f
scontext_undefined			f	f
mscontext_undefined			f	f
amo_trigger			f	f
no_hit			f	f
no_sselect_2			f	f
trigger_num	0	255	4	4

mask_tselect			f	f
tinfo	0	0x100ffff	0x100807c	0x100807c
tinfo0	0	0xffffffff	0x100807c	0x100807c
tinfo1	0	0xffffffff	0x100807c	0x100807c
tinfo2	0	0xffffffff	0x100807c	0x100807c
tinfo3	0	0xffffffff	0x100807c	0x100807c
tinfo4	0	0xffffffff	0x100807c	0x100807c
tinfo5	0	0xffffffff	0x100807c	0x100807c
tinfo6	0	0xffffffff	0x100807c	0x100807c
tinfo7	0	0xffffffff	0x100807c	0x100807c
tinfo8	0	0xffffffff	0x100807c	0x100807c
tinfo9	0	0xffffffff	0x100807c	0x100807c
tinfo10	0	0xffffffff	0x100807c	0x100807c
tinfo11	0	0xffffffff	0x100807c	0x100807c
tinfo12	0	0xffffffff	0x100807c	0x100807c
tinfo13	0	0xffffffff	0x100807c	0x100807c
tinfo14	0	0xffffffff	0x100807c	0x100807c
tinfo15	0	0xffffffff	0x100807c	0x100807c
trigger_match	1	0xffff	0x333f	0x333f
mcontext_bits	0	64	13	13
scontext_bits	0	64	34	34
mvalue_bits	0	13	13	13
svalue_bits	0	34	34	34
mcontrol_maskmax	0	63	63	63
chain_tval			first	first
mcontrol_addl_comp			f	f
PMP Configuration				
PMP_grain	0	29	0	0
PMP_registers	0	64	16	16
PMP_csrs	0	64	0	0
PMP_max_page	0	0xffffffff	0	0
PMP_decompose			f	f
PMP_undefined			f	f
PMP_R0W1			RWX_00X	RWX_00X
PMP_A_RSVD			PMP_A_NONE	PMP_A_NONE
PMP_maskparams			f	f
PMP_initialparams			f	f
Sspmp			f	f
mpmpdeleg_min	0	127	0	0
mpmpdeleg_max	0	127	0	0
Other_Extensions				
Svnapot_page_mask	0	0xffffffffffffff	0	0
Smstateen			f	f
Ssstateen			f	f
Smcsrind			f	f

Sstc			f	f
Sscofpmf			f	f
Ssdbltrp			f	f
Smdbltrp			f	f
Smcntrpmf			f	f
Smcdeleg			f	f
Svpbmt			f	f
Ssnpm			none	none
Smnpm			none	none
Smmpm			none	none
Svinval			f	f
Svadu			f	f
Ssqosid			f	f
Zihintntl			f	f
Zihintpause			t	t
Zicond			f	f
Zicsr			t	t
Zifencei			t	t
Zicbom			f	f
Zicbop			f	f
Zicboz			f	f
Zimop			f	f
Zicfiss			f	f
Zicfilp			f	f
Zibi			f	f
Zaamo			t	t
Zalrsc			t	t
Zacas			f	f
Zabha			f	f
Zawrs			f	f
Zmmul			f	f
Bit_Manipulation_Extension				
misa_B_Zba_Zbb_Zbs			f	f
Zba			t	t
Zbb			t	t
Zbc			t	t
Zbe			t	t
Zbf			t	t
Zbm			t	t
Zbp			t	t
Zbr			t	t
Zbs			t	t
Zbt			t	t
CSR_Defaults				
mvendorid	0	0xffffffffffffffff	0	0

marchid	0	0xffffffffffffffff	0	0
mimpid	0	0xffffffffffffffff	0	0
mhartid	0	0xffffffffffffffff	0	0
mconfigptr	0	0xffffffffffffffff	0	0
mtvec	0	0xffffffffffffffff	0	0
mseccfg	0	0xffffffffffffffff	0	0
menvcfg	0	0xffffffffffffffff	0	0
AIA_Interrupts				
Smaia			f	f

Table 8.27: Parameter values and limits

Chapter 9

Execution Modes

Mode	Code	Description
User	0	User mode
Supervisor	1	Supervisor mode
Machine	3	Machine mode

Table 9.1: Modes implemented in: Hart

Chapter 10

Exceptions

Exception	Code	Description
InstructionAddressMisaligned	0	Fetch from unaligned address
InstructionAccessFault	1	No access permission for fetch
IllegalInstruction	2	Undecoded, unimplemented or disabled instruction
Breakpoint	3	EBREAK instruction executed
LoadAddressMisaligned	4	Load from unaligned address
LoadAccessFault	5	No access permission for load
StoreAMOAddressMisaligned	6	Store/atomic memory operation at unaligned address
StoreAMOAccessFault	7	No access permission for store/atomic memory operation
EnvironmentCallFromUMode	8	ECALL instruction executed in User mode
EnvironmentCallFromSMode	9	ECALL instruction executed in Supervisor mode
EnvironmentCallFromMMode	11	ECALL instruction executed in Machine mode
InstructionPageFault	12	Page fault at fetch address
LoadPageFault	13	Page fault at load address
StoreAMOPageFault	15	Page fault at store/atomic memory operation address
SoftwareCheck	18	Software check
HardwareError	19	Hardware Error
SSWInterrupt	65	Supervisor software interrupt
MSWInterrupt	67	Machine software interrupt
STimerInterrupt	69	Supervisor timer interrupt
MTimerInterrupt	71	Machine timer interrupt
SExternalInterrupt	73	Supervisor external interrupt
MExternalInterrupt	75	Machine external interrupt
GenericNMI	4294967295	Generic NMI

Table 10.1: Exceptions implemented in: Hart

Chapter 11

Hierarchy of the model

A CPU core may be configured to instance many processors of a Symmetrical Multi Processor (SMP). A CPU core may also have sub elements within a processor, for example hardware threading blocks.
OVP processor models can be written to include SMP blocks and to have many levels of hierarchy. Some OVP CPU models may have a fixed hierarchy, and some may be configured by settings in a configuration register. Please see the register definitions of this model.
This model documentation shows the settings and hierarchy of the default settings for this model variant.

11.1 Level 1: Hart

This level in the model hierarchy has 9 commands.
This level in the model hierarchy has 7 register groups:

Group name	Registers
Core	33
Floating_point	32
Vector	32
User_Control_and_Status	42
Supervisor_Control_and_Status	13
Machine_Control_and_Status	158
Integration_support	64

Table 11.1: Register groups

This level in the model hierarchy has no children.

Chapter 12

Model Commands

A Processor model can implement one or more **Model Commands** available to be invoked from the simulator command line, from the OP API or from the Imperas Multiprocessor Debugger.

12.1 Level 1: Hart

12.1.1 addLRSCRegion

Add LR/SC region with the given grain size

Argument	Type	Description
-grainSizeM1	Uns64	grain size, minus 1 (e.g. specify 63 for 64-bit reservation grain)
-hi	Uns64	high address
-lo	Uns64	low address

Table 12.1: addLRSCRegion command arguments

12.1.2 debugflags

show or modify the processor debug flags

Argument	Type	Description
-get	Boolean	print current processor flags value
-mask	Boolean	print valid debug flag bits
-set	Int32	new processor flags (only flags 0x00000006 can be modified)

Table 12.2: debugflags command arguments

12.1.3 dumpTLB

12.1.3.1 Argument description

Show TLB contents

12.1.4 getCSRIndex

Return index for a named CSR (or -1 if no matching CSR)

Argument	Type	Description
-name	String	CSR name

Table 12.3: getCSRIndex command arguments

12.1.5 isync

specify instruction address range for synchronous execution

Argument	Type	Description
-addresshi	Uns64	end address of synchronous execution range
-addresslo	Uns64	start address of synchronous execution range

Table 12.4: isync command arguments

12.1.6 itrace

enable or disable instruction tracing

Argument	Type	Description
-access	String	show memory accesses by this instruction. Argument can be any combination of X (execute), A (load or store access) and S (system)
-after	Uns64	apply after this many instructions
-enable	Boolean	enable instruction tracing
-full	Boolean	turn on all trace features
-instructioncount	Boolean	include the instruction number in each trace
-memory	String	(Alias for access). show memory accesses by this instruction. Argument can be any combination of X (execute), A (load or store access) and S (system)
-mode	Boolean	show processor mode changes
-off	Boolean	disable instruction tracing
-on	Boolean	enable instruction tracing
-processorname	Boolean	Include processor name in all trace lines
-registerchange	Boolean	show registers changed by this instruction
-registers	Boolean	show registers after each trace

Table 12.5: itrace command arguments

12.1.7 listCSRs

12.1.7.1 Argument description

List all CSRs in index order

12.1.8 resetLRSCRegion

Reset LR/SC region (use configured lr_sc_grain in this range)

Argument	Type	Description
-hi	Uns64	high address
-lo	Uns64	low address

Table 12.6: resetLRSCRegion command arguments

12.1.9 setPMA

Set PMA region permissions and legal access sizes

Argument	Type	Description
-attributes	String	region attributes (string containing r, w, x, a, A, P, 1, 2, 4 or 8)
-hi	Uns64	high address
-lo	Uns64	low address

Table 12.7: setPMA command arguments

Chapter 13

Registers

13.1 Level 1: Hart

13.1.1 Core

Registers at level:1, type:Hart group:Core

Name	Bits	Initial-Hex	RW	Description
zero	64	0	r-	
ra	64	0	rw	
sp	64	0	rw	stack pointer
gp	64	0	rw	
tp	64	0	rw	
t0	64	0	rw	
t1	64	0	rw	
t2	64	0	rw	
s0	64	0	rw	
s1	64	0	rw	
a0	64	0	rw	
a1	64	0	rw	
a2	64	0	rw	
a3	64	0	rw	
a4	64	0	rw	
a5	64	0	rw	
a6	64	0	rw	
a7	64	0	rw	
s2	64	0	rw	
s3	64	0	rw	
s4	64	0	rw	
s5	64	0	rw	
s6	64	0	rw	
s7	64	0	rw	
s8	64	0	rw	
s9	64	0	rw	
s10	64	0	rw	
s11	64	0	rw	
t3	64	0	rw	
t4	64	0	rw	
t5	64	0	rw	
t6	64	0	rw	
pc	64	0	rw	program counter

Table 13.1: Registers at level 1, type:Hart group:Core

13.1.2 Floating_point

Registers at level:1, type:Hart group:Floating_point

Name	Bits	Initial-Hex	RW	Description
ft0	64	0	rw	
ft1	64	0	rw	
ft2	64	0	rw	
ft3	64	0	rw	
ft4	64	0	rw	
ft5	64	0	rw	
ft6	64	0	rw	
ft7	64	0	rw	
fs0	64	0	rw	
fs1	64	0	rw	
fa0	64	0	rw	
fa1	64	0	rw	
fa2	64	0	rw	
fa3	64	0	rw	
fa4	64	0	rw	
fa5	64	0	rw	
fa6	64	0	rw	
fa7	64	0	rw	
fs2	64	0	rw	
fs3	64	0	rw	
fs4	64	0	rw	
fs5	64	0	rw	
fs6	64	0	rw	
fs7	64	0	rw	
fs8	64	0	rw	
fs9	64	0	rw	
fs10	64	0	rw	
fs11	64	0	rw	
ft8	64	0	rw	
ft9	64	0	rw	
ft10	64	0	rw	
ft11	64	0	rw	

Table 13.2: Registers at level 1, type:Hart group:Floating_point

13.1.3 Vector

Registers at level:1, type:Hart group:Vector

Name	Bits	Initial-Hex	RW	Description
v0	512	-	rw	
v1	512	-	rw	
v2	512	-	rw	
v3	512	-	rw	
v4	512	-	rw	
v5	512	-	rw	
v6	512	-	rw	

v7	512	-	rw	
v8	512	-	rw	
v9	512	-	rw	
v10	512	-	rw	
v11	512	-	rw	
v12	512	-	rw	
v13	512	-	rw	
v14	512	-	rw	
v15	512	-	rw	
v16	512	-	rw	
v17	512	-	rw	
v18	512	-	rw	
v19	512	-	rw	
v20	512	-	rw	
v21	512	-	rw	
v22	512	-	rw	
v23	512	-	rw	
v24	512	-	rw	
v25	512	-	rw	
v26	512	-	rw	
v27	512	-	rw	
v28	512	-	rw	
v29	512	-	rw	
v30	512	-	rw	
v31	512	-	rw	

Table 13.3: Registers at level 1, type:Hart group:Vector

13.1.4 User_Control_and_Status

Registers at level:1, type:Hart group:User_Control_and_Status

Name	Bits	Initial-Hex	RW	Description
fflags	64	0	rw	Floating-Point Flags
frm	64	0	rw	Floating-Point Rounding Mode
fcsr	64	0	rw	Floating-Point Control and Status
vstart	64	0	rw	Vector Start Index
vxsat	64	0	rw	Fixed-Point Saturate Flag
vxrm	64	0	rw	Fixed-Point Rounding Mode
vcsr	64	0	rw	Vector Control and Status
cycle	64	0	r-	Cycle Counter
time	64	0	r-	Timer
instret	64	0	r-	Instructions Retired
hpmcounter3	64	0	r-	Performance Monitor Counter 3
hpmcounter4	64	0	r-	Performance Monitor Counter 4
hpmcounter5	64	0	r-	Performance Monitor Counter 5
hpmcounter6	64	0	r-	Performance Monitor Counter 6
hpmcounter7	64	0	r-	Performance Monitor Counter 7
hpmcounter8	64	0	r-	Performance Monitor Counter 8
hpmcounter9	64	0	r-	Performance Monitor Counter 9
hpmcounter10	64	0	r-	Performance Monitor Counter 10
hpmcounter11	64	0	r-	Performance Monitor Counter 11
hpmcounter12	64	0	r-	Performance Monitor Counter 12
hpmcounter13	64	0	r-	Performance Monitor Counter 13
hpmcounter14	64	0	r-	Performance Monitor Counter 14

hpmcounter15	64	0	r-	Performance Monitor Counter 15
hpmcounter16	64	0	r-	Performance Monitor Counter 16
hpmcounter17	64	0	r-	Performance Monitor Counter 17
hpmcounter18	64	0	r-	Performance Monitor Counter 18
hpmcounter19	64	0	r-	Performance Monitor Counter 19
hpmcounter20	64	0	r-	Performance Monitor Counter 20
hpmcounter21	64	0	r-	Performance Monitor Counter 21
hpmcounter22	64	0	r-	Performance Monitor Counter 22
hpmcounter23	64	0	r-	Performance Monitor Counter 23
hpmcounter24	64	0	r-	Performance Monitor Counter 24
hpmcounter25	64	0	r-	Performance Monitor Counter 25
hpmcounter26	64	0	r-	Performance Monitor Counter 26
hpmcounter27	64	0	r-	Performance Monitor Counter 27
hpmcounter28	64	0	r-	Performance Monitor Counter 28
hpmcounter29	64	0	r-	Performance Monitor Counter 29
hpmcounter30	64	0	r-	Performance Monitor Counter 30
hpmcounter31	64	0	r-	Performance Monitor Counter 31
vl	64	0	r-	Vector Length
vtype	64	0	r-	Vector Type
vlenb	64	40	r-	Vector Length in Bytes

Table 13.4: Registers at level 1, type:Hart group:User_Control_and_Status

13.1.5 Supervisor_Control_and_Status

Registers at level:1, type:Hart group:Supervisor_Control_and_Status

Name	Bits	Initial-Hex	RW	Description
sstatus	64	2 00000000	rw	Supervisor Status
sie	64	0	rw	Supervisor Interrupt Enable
stvec	64	0	rw	Supervisor Trap-Vector Base-Address
scounteren	64	0	rw	Supervisor Counter Enable
senvcfg	64	0	rw	Supervisor Environment Configuration
sscratch	64	0	rw	Supervisor Scratch
sepc	64	0	rw	Supervisor Exception Program Counter
scause	64	0	rw	Supervisor Cause
stval	64	0	rw	Supervisor Trap Value
sip	64	0	rw	Supervisor Interrupt Pending
satp	64	0	rw	Supervisor Address Translation and Protection
scontext	64	0	rw	Trigger Supervisor Context
mscontext	64	0	rw	Trigger Machine Context Alias

Table 13.5: Registers at level 1, type:Hart group:Supervisor_Control_and_Status

13.1.6 Machine_Control_and_Status

Registers at level:1, type:Hart group:Machine_Control_and_Status

Name	Bits	Initial-Hex	RW	Description
mstatus	64	a 00000000	rw	Machine Status
misa	64	80000000 0034112d	rw	ISA and Extensions
medeleg	64	0	rw	Machine Exception Delegation
mideleg	64	0	rw	Machine Interrupt Delegation
mie	64	0	rw	Machine Interrupt Enable

mtvec	64	0	rw	Machine Trap-Vector Base-Address
mcounteren	64	0	rw	Machine Counter Enable
menvcfg	64	0	rw	Machine Environment Configuration
mcountinhibit	64	0	rw	Machine Counter Inhibit
mhpmevent3	64	0	rw	Machine Performance Monitor Event Select 3
mhpmevent4	64	0	rw	Machine Performance Monitor Event Select 4
mhpmevent5	64	0	rw	Machine Performance Monitor Event Select 5
mhpmevent6	64	0	rw	Machine Performance Monitor Event Select 6
mhpmevent7	64	0	rw	Machine Performance Monitor Event Select 7
mhpmevent8	64	0	rw	Machine Performance Monitor Event Select 8
mhpmevent9	64	0	rw	Machine Performance Monitor Event Select 9
mhpmevent10	64	0	rw	Machine Performance Monitor Event Select 10
mhpmevent11	64	0	rw	Machine Performance Monitor Event Select 11
mhpmevent12	64	0	rw	Machine Performance Monitor Event Select 12
mhpmevent13	64	0	rw	Machine Performance Monitor Event Select 13
mhpmevent14	64	0	rw	Machine Performance Monitor Event Select 14
mhpmevent15	64	0	rw	Machine Performance Monitor Event Select 15
mhpmevent16	64	0	rw	Machine Performance Monitor Event Select 16
mhpmevent17	64	0	rw	Machine Performance Monitor Event Select 17
mhpmevent18	64	0	rw	Machine Performance Monitor Event Select 18
mhpmevent19	64	0	rw	Machine Performance Monitor Event Select 19
mhpmevent20	64	0	rw	Machine Performance Monitor Event Select 20
mhpmevent21	64	0	rw	Machine Performance Monitor Event Select 21
mhpmevent22	64	0	rw	Machine Performance Monitor Event Select 22
mhpmevent23	64	0	rw	Machine Performance Monitor Event Select 23
mhpmevent24	64	0	rw	Machine Performance Monitor Event Select 24
mhpmevent25	64	0	rw	Machine Performance Monitor Event Select 25
mhpmevent26	64	0	rw	Machine Performance Monitor Event Select 26
mhpmevent27	64	0	rw	Machine Performance Monitor Event Select 27
mhpmevent28	64	0	rw	Machine Performance Monitor Event Select 28
mhpmevent29	64	0	rw	Machine Performance Monitor Event Select 29
mhpmevent30	64	0	rw	Machine Performance Monitor Event Select 30
mhpmevent31	64	0	rw	Machine Performance Monitor Event Select 31
mscratch	64	0	rw	Machine Scratch
mepc	64	0	rw	Machine Exception Program Counter
mcause	64	0	rw	Machine Cause
mtval	64	0	rw	Machine Trap Value
mip	64	0	rw	Machine Interrupt Pending
pmpcfg0	64	0	rw	Physical Memory Protection Configuration 0
pmpcfg2	64	0	rw	Physical Memory Protection Configuration 2
pmpcfg4	64	0	rw	Physical Memory Protection Configuration 4
pmpcfg6	64	0	rw	Physical Memory Protection Configuration 6
pmpcfg8	64	0	rw	Physical Memory Protection Configuration 8
pmpcfg10	64	0	rw	Physical Memory Protection Configuration 10
pmpcfg12	64	0	rw	Physical Memory Protection Configuration 12
pmpcfg14	64	0	rw	Physical Memory Protection Configuration 14
pmpaddr0	64	0	rw	Physical Memory Protection Address 0
pmpaddr1	64	0	rw	Physical Memory Protection Address 1
pmpaddr2	64	0	rw	Physical Memory Protection Address 2
pmpaddr3	64	0	rw	Physical Memory Protection Address 3
pmpaddr4	64	0	rw	Physical Memory Protection Address 4
pmpaddr5	64	0	rw	Physical Memory Protection Address 5
pmpaddr6	64	0	rw	Physical Memory Protection Address 6
pmpaddr7	64	0	rw	Physical Memory Protection Address 7
pmpaddr8	64	0	rw	Physical Memory Protection Address 8
pmpaddr9	64	0	rw	Physical Memory Protection Address 9

pmpaddr10	64	0	rw	Physical Memory Protection Address 10
pmpaddr11	64	0	rw	Physical Memory Protection Address 11
pmpaddr12	64	0	rw	Physical Memory Protection Address 12
pmpaddr13	64	0	rw	Physical Memory Protection Address 13
pmpaddr14	64	0	rw	Physical Memory Protection Address 14
pmpaddr15	64	0	rw	Physical Memory Protection Address 15
pmpaddr16	64	0	rw	Physical Memory Protection Address 16
pmpaddr17	64	0	rw	Physical Memory Protection Address 17
pmpaddr18	64	0	rw	Physical Memory Protection Address 18
pmpaddr19	64	0	rw	Physical Memory Protection Address 19
pmpaddr20	64	0	rw	Physical Memory Protection Address 20
pmpaddr21	64	0	rw	Physical Memory Protection Address 21
pmpaddr22	64	0	rw	Physical Memory Protection Address 22
pmpaddr23	64	0	rw	Physical Memory Protection Address 23
pmpaddr24	64	0	rw	Physical Memory Protection Address 24
pmpaddr25	64	0	rw	Physical Memory Protection Address 25
pmpaddr26	64	0	rw	Physical Memory Protection Address 26
pmpaddr27	64	0	rw	Physical Memory Protection Address 27
pmpaddr28	64	0	rw	Physical Memory Protection Address 28
pmpaddr29	64	0	rw	Physical Memory Protection Address 29
pmpaddr30	64	0	rw	Physical Memory Protection Address 30
pmpaddr31	64	0	rw	Physical Memory Protection Address 31
pmpaddr32	64	0	rw	Physical Memory Protection Address 32
pmpaddr33	64	0	rw	Physical Memory Protection Address 33
pmpaddr34	64	0	rw	Physical Memory Protection Address 34
pmpaddr35	64	0	rw	Physical Memory Protection Address 35
pmpaddr36	64	0	rw	Physical Memory Protection Address 36
pmpaddr37	64	0	rw	Physical Memory Protection Address 37
pmpaddr38	64	0	rw	Physical Memory Protection Address 38
pmpaddr39	64	0	rw	Physical Memory Protection Address 39
pmpaddr40	64	0	rw	Physical Memory Protection Address 40
pmpaddr41	64	0	rw	Physical Memory Protection Address 41
pmpaddr42	64	0	rw	Physical Memory Protection Address 42
pmpaddr43	64	0	rw	Physical Memory Protection Address 43
pmpaddr44	64	0	rw	Physical Memory Protection Address 44
pmpaddr45	64	0	rw	Physical Memory Protection Address 45
pmpaddr46	64	0	rw	Physical Memory Protection Address 46
pmpaddr47	64	0	rw	Physical Memory Protection Address 47
pmpaddr48	64	0	rw	Physical Memory Protection Address 48
pmpaddr49	64	0	rw	Physical Memory Protection Address 49
pmpaddr50	64	0	rw	Physical Memory Protection Address 50
pmpaddr51	64	0	rw	Physical Memory Protection Address 51
pmpaddr52	64	0	rw	Physical Memory Protection Address 52
pmpaddr53	64	0	rw	Physical Memory Protection Address 53
pmpaddr54	64	0	rw	Physical Memory Protection Address 54
pmpaddr55	64	0	rw	Physical Memory Protection Address 55
pmpaddr56	64	0	rw	Physical Memory Protection Address 56
pmpaddr57	64	0	rw	Physical Memory Protection Address 57
pmpaddr58	64	0	rw	Physical Memory Protection Address 58
pmpaddr59	64	0	rw	Physical Memory Protection Address 59
pmpaddr60	64	0	rw	Physical Memory Protection Address 60
pmpaddr61	64	0	rw	Physical Memory Protection Address 61
pmpaddr62	64	0	rw	Physical Memory Protection Address 62
pmpaddr63	64	0	rw	Physical Memory Protection Address 63
tselect	64	0	rw	Trigger Register Select

tdata1	64	f0000000 00000000	rw	Trigger Data 1
tdata2	64	0	rw	Trigger Data 2
tdata3	64	0	rw	Trigger Data 3
tinfo	64	100807c	rw	Trigger Info
tcontrol	64	0	rw	Trigger Control
mcontext	64	0	rw	Trigger Machine Context
mcycle	64	0	rw	Machine Cycle Counter
minstret	64	0	rw	Machine Instructions Retired
mhpmcounter3	64	0	rw	Machine Performance Monitor Counter 3
mhpmcounter4	64	0	rw	Machine Performance Monitor Counter 4
mhpmcounter5	64	0	rw	Machine Performance Monitor Counter 5
mhpmcounter6	64	0	rw	Machine Performance Monitor Counter 6
mhpmcounter7	64	0	rw	Machine Performance Monitor Counter 7
mhpmcounter8	64	0	rw	Machine Performance Monitor Counter 8
mhpmcounter9	64	0	rw	Machine Performance Monitor Counter 9
mhpmcounter10	64	0	rw	Machine Performance Monitor Counter 10
mhpmcounter11	64	0	rw	Machine Performance Monitor Counter 11
mhpmcounter12	64	0	rw	Machine Performance Monitor Counter 12
mhpmcounter13	64	0	rw	Machine Performance Monitor Counter 13
mhpmcounter14	64	0	rw	Machine Performance Monitor Counter 14
mhpmcounter15	64	0	rw	Machine Performance Monitor Counter 15
mhpmcounter16	64	0	rw	Machine Performance Monitor Counter 16
mhpmcounter17	64	0	rw	Machine Performance Monitor Counter 17
mhpmcounter18	64	0	rw	Machine Performance Monitor Counter 18
mhpmcounter19	64	0	rw	Machine Performance Monitor Counter 19
mhpmcounter20	64	0	rw	Machine Performance Monitor Counter 20
mhpmcounter21	64	0	rw	Machine Performance Monitor Counter 21
mhpmcounter22	64	0	rw	Machine Performance Monitor Counter 22
mhpmcounter23	64	0	rw	Machine Performance Monitor Counter 23
mhpmcounter24	64	0	rw	Machine Performance Monitor Counter 24
mhpmcounter25	64	0	rw	Machine Performance Monitor Counter 25
mhpmcounter26	64	0	rw	Machine Performance Monitor Counter 26
mhpmcounter27	64	0	rw	Machine Performance Monitor Counter 27
mhpmcounter28	64	0	rw	Machine Performance Monitor Counter 28
mhpmcounter29	64	0	rw	Machine Performance Monitor Counter 29
mhpmcounter30	64	0	rw	Machine Performance Monitor Counter 30
mhpmcounter31	64	0	rw	Machine Performance Monitor Counter 31
mvendorid	64	0	r-	Vendor ID
marchid	64	0	r-	Architecture ID
mimpid	64	0	r-	Implementation ID
mhartid	64	0	r-	Hardware Thread ID
mconfigptr	64	0	r-	Configuration Data Structure

Table 13.6: Registers at level 1, type:Hart group:Machine_Control_and_Status

13.1.7 Integration_support

Registers at level:1, type:Hart group:Integration_support

Name	Bits	Initial-Hex	RW	Description
LRSCAddress	64	ffffff ffffff	rw	LR/SC active lock address
LRSCSize	8	0	rw	LR/SC active access byte size
commercial	8	0	r-	Commercial feature in use

PTWStage	8	0	r-	PTW active stage from memory callback context (0:none 1:HS 2:VS 3:G)
PTWInputAddr	64	0	r-	PTW input address from memory callback context
PTWLevel	8	0	r-	PTW active level from memory callback context
ASYNCPE	8	0	r-	Asynchronous Event Pending & Enabled
HaltReason	32	0	r-	Bit field indicating halt reason
mask_pmpcfg0	64	ffffffff ffffffff	r-	Write mask for pmpcfg0
mask_pmpcfg2	64	ffffffff ffffffff	r-	Write mask for pmpcfg2
mask_pmpaddr0	64	ffffffff ffffffff	r-	Write mask for pmpaddr0
mask_pmpaddr1	64	ffffffff ffffffff	r-	Write mask for pmpaddr1
mask_pmpaddr2	64	ffffffff ffffffff	r-	Write mask for pmpaddr2
mask_pmpaddr3	64	ffffffff ffffffff	r-	Write mask for pmpaddr3
mask_pmpaddr4	64	ffffffff ffffffff	r-	Write mask for pmpaddr4
mask_pmpaddr5	64	ffffffff ffffffff	r-	Write mask for pmpaddr5
mask_pmpaddr6	64	ffffffff ffffffff	r-	Write mask for pmpaddr6
mask_pmpaddr7	64	ffffffff ffffffff	r-	Write mask for pmpaddr7
mask_pmpaddr8	64	ffffffff ffffffff	r-	Write mask for pmpaddr8
mask_pmpaddr9	64	ffffffff ffffffff	r-	Write mask for pmpaddr9
mask_pmpaddr10	64	ffffffff ffffffff	r-	Write mask for pmpaddr10
mask_pmpaddr11	64	ffffffff ffffffff	r-	Write mask for pmpaddr11
mask_pmpaddr12	64	ffffffff ffffffff	r-	Write mask for pmpaddr12
mask_pmpaddr13	64	ffffffff ffffffff	r-	Write mask for pmpaddr13
mask_pmpaddr14	64	ffffffff ffffffff	r-	Write mask for pmpaddr14
mask_pmpaddr15	64	ffffffff ffffffff	r-	Write mask for pmpaddr15
PTWBankSelect	8	0	rw	select PTW bank (0 is stage 1, 1 is stage 2, 2-6 are stage 2 walks initiated by stage 1 level 0-4 entry lookups, respectively)
PTWBankValid	8	0	r-	bitmask of valid banks (0x01 is stage 1, 0x02 is stage 2, 0x04-0x40 are stage 2 walks initiated by stage 1 level 0-4 entry lookups, respectively)
PTWAddressValid	8	0	r-	bitmask of valid bits for each of PTWAddressL0...PTWAddressL4, PTWBase, PTWInput and PTWOutput in current bank
PTWValueValid	8	0	r-	bitmask of valid bits for each of PTWValueL0...PTWValueL4 in current bank
PTWAddressL0	64	0	r-	current bank PTW address, level 0
PTWAddressL1	64	0	r-	current bank PTW address, level 1
PTWAddressL2	64	0	r-	current bank PTW address, level 2
PTWAddressL3	64	0	r-	current bank PTW address, level 3
PTWAddressL4	64	0	r-	current bank PTW address, level 4
PTWValueL0	64	0	r-	current bank PTW value, level 0
PTWValueL1	64	0	r-	current bank PTW value, level 1
PTWValueL2	64	0	r-	current bank PTW value, level 2
PTWValueL3	64	0	r-	current bank PTW value, level 3
PTWValueL4	64	0	r-	current bank PTW value, level 4
PTWBase	64	0	r-	current bank PTW table base address
PTWInput	64	0	r-	current bank PTW input address
PTWOutput	64	0	r-	current bank PTW output address
PTWPgSize	64	0	r-	current bank PTW page size (Valid only when PTWOutput is valid)
PTWUD	64	0	-w	perform [H]U-mode page table walk for the given data address
PTWUX	64	0	-w	perform [H]U-mode page table walk for the given execute address
PTWSD	64	0	-w	perform [H]S-mode page table walk for the given data address
PTWSX	64	0	-w	perform [H]S-mode page table walk for the given execute address
pmp0cfg0	8	0	r-	
pmp1cfg0	8	0	r-	
pmp2cfg0	8	0	r-	

pmp3cfg0	8	0	r-	
pmp4cfg0	8	0	r-	
pmp5cfg0	8	0	r-	
pmp6cfg0	8	0	r-	
pmp7cfg0	8	0	r-	
pmp8cfg2	8	0	r-	
pmp9cfg2	8	0	r-	
pmp10cfg2	8	0	r-	
pmp11cfg2	8	0	r-	
pmp12cfg2	8	0	r-	
pmp13cfg2	8	0	r-	
pmp14cfg2	8	0	r-	
pmp15cfg2	8	0	r-	

Table 13.7: Registers at level 1, type:Hart group:Integration_support